Microsoft Power Platform

Lifecycle Mastery: Managing Power Platform Assets in the Age of Agents

Step 1: Get the labs

https://aka.ms/PPCC26-ALM

Step 2: Get a login

https://aka.ms/PPCCWorkshopAccounts

Microsoft Power Platform

Presented by:



Marcel Ferreira
Principal Product Manager
Developer experiences



Prabhat Pandey
Principal Engineering Manager
DevOps



Casey Burke
Principal Product Manager
DevOps



Yash Patel
Senior Program Manager
Power CAT



Ashish Choudhary
Principal Engineering Manager
ProDev



Priyanshu Agrawal
Principal Software Engineer
ProDev

Neeraj Nandwana
Principal Product Manager
ProDev

Microsoft Power Platform

Proctors in the room:



Brian Gregor
Power Platform
Global Black Belt



Arjun Mayur
Product Manager II
DevOps / Monitor

Greg Lindhorst
Principal Product
Manager



Hei Wang Chan
Principal Engineer
DevOps



Jay Bujula
Senior Engineer
DevOps



Kartik K R
Senior Engineer
DevOps

Rahul Ranjit
Kannathusseril
Principal PM
Power CAT

Microsoft Power Platform

Step 1: Get the labs

https://aka.ms/PPCC26-ALM

Step 2: Get a login

https://aka.ms/PPCCWorkshopAccounts

Please be respectful and remember:

- No jerks
- The presenters volunteered time and resources for this workshop
- Despite our best efforts to prepare, there can be hiccups with fresh content
- This is not a Microsoft conference. Please contact the event organizers for any issues unrelated to the content, products, and features covered in this workshop.

Agenda

Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

Microsoft Power Platform

Solutions and ALM 101

Step 1: Get the labs

https://aka.ms/PPCC26-ALM

Step 2: Get a login

https://aka.ms/PPCCWorkshopAccounts

Show of hands:

How long have you used DevOps in Power Platform?

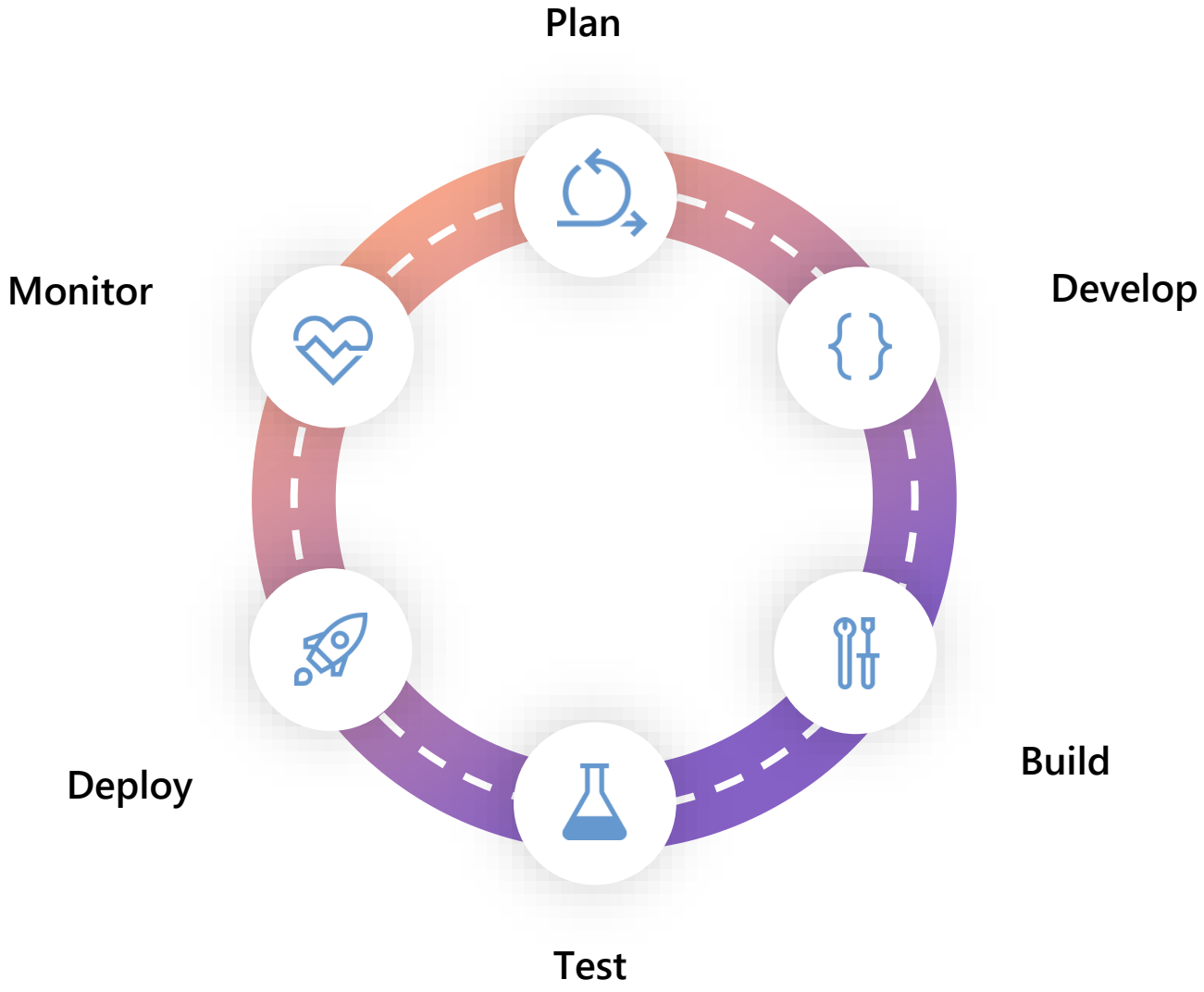
New | < 2 yrs | 2–5 yrs | 5–10 yrs | 10+ yrs

3x usage

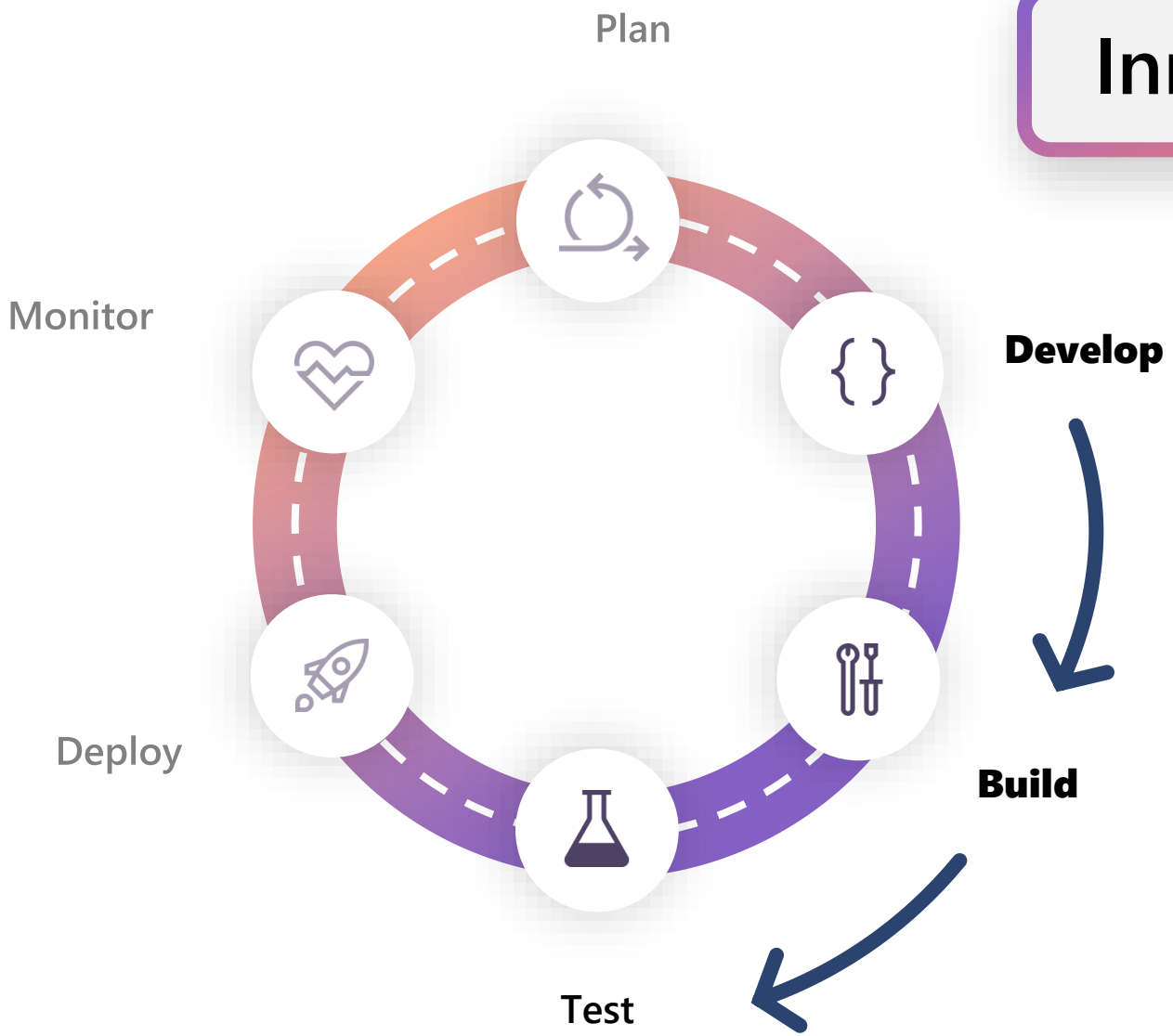
Solutions using ALM have
3x more usage on average

What is ALM?

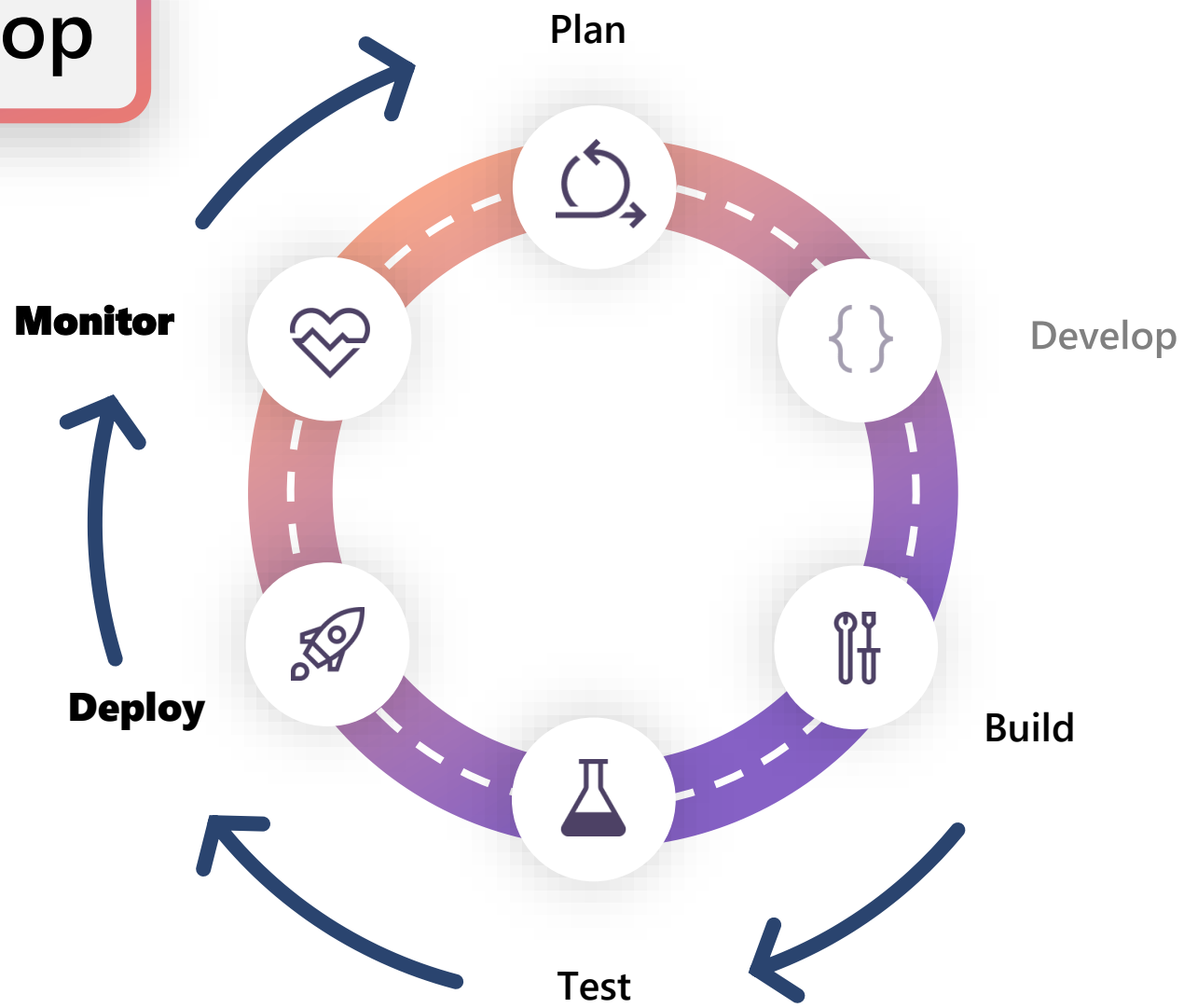
*DevOps is the union of **people**, **process**, and **products** to enable **continuous delivery** of **value** to end users*


Plan
Develop
Build
Test
Deploy
Monitor

Inner Loop



Outer Loop



ALM fundamentals in Power Platform

Environments

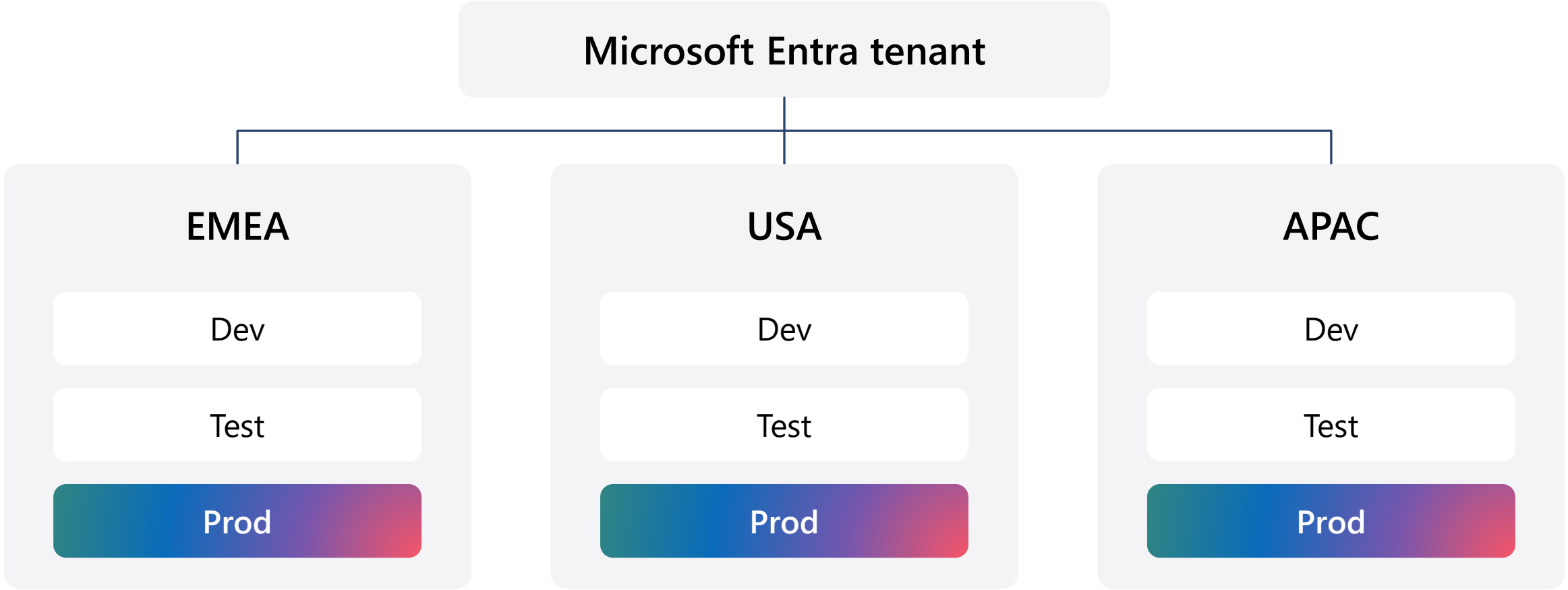
Default environment = personal productivity => **not for ALM**

Dev, test, and production recommended for most projects

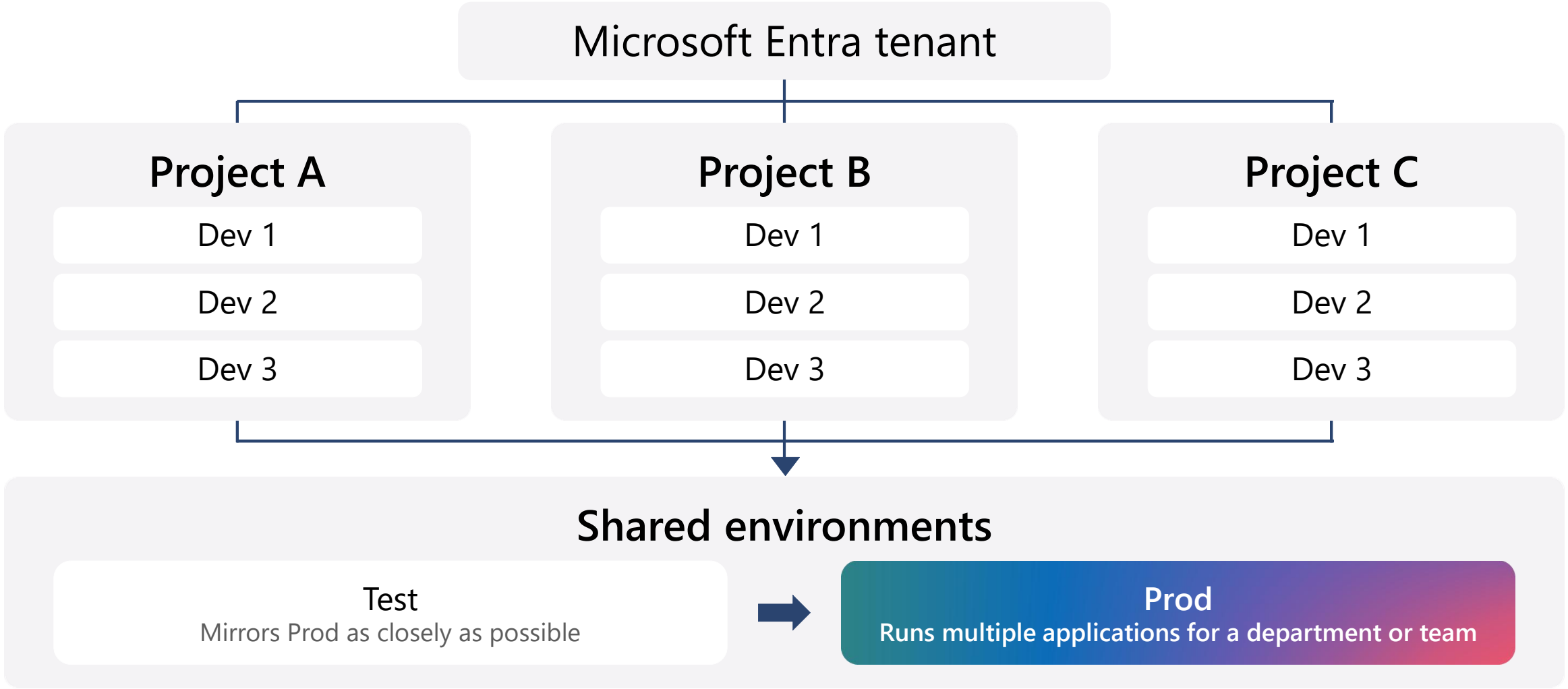
Never develop in **test** or **prod** – use restrictive security roles and deploy with a service principle (vs granting maker permissions)

Environments can be **grouped** together to **apply policies uniformly** (e.g. developer environments)

Your environment strategy



Environment strategy



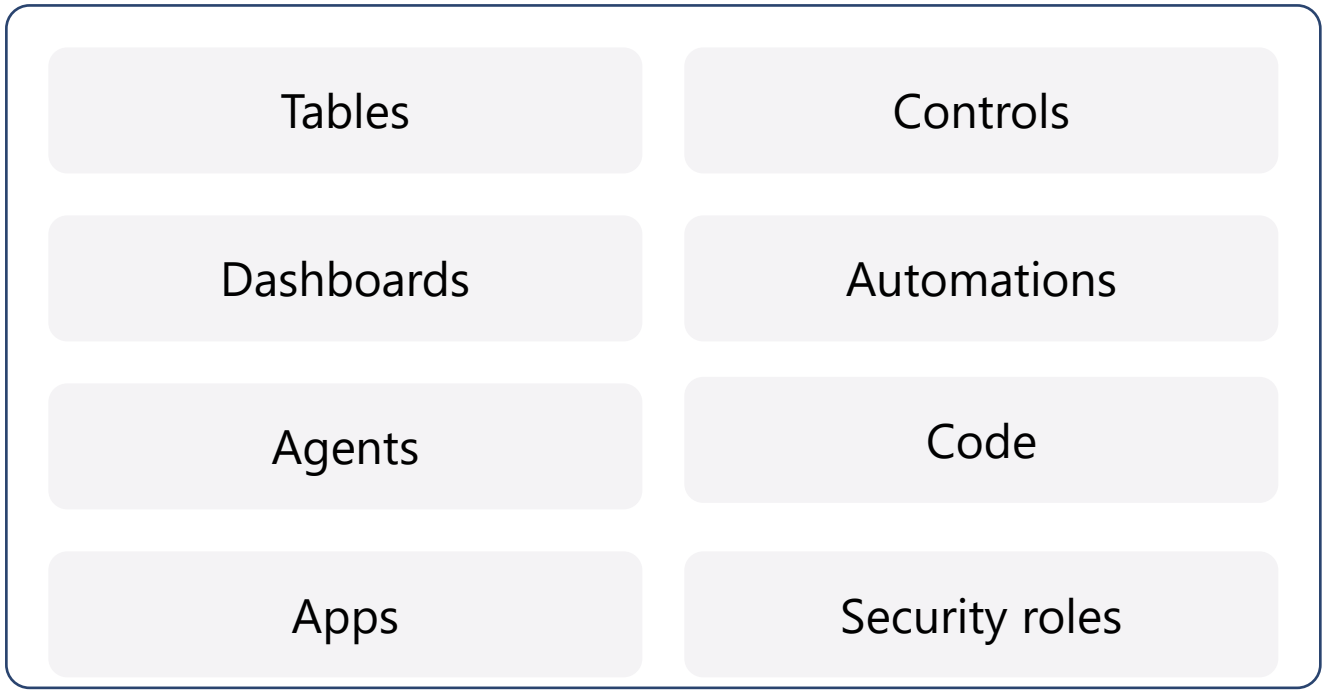
What is a solution?

Container of objects that make up your application

Exported (built) as an artifact that's deployed across environments and tenants

There's two default solutions – **don't develop using these!**

Create a **custom** solution – create/edit everything from this entry point



Solutions are your customizations - not business data

Preferred Solution

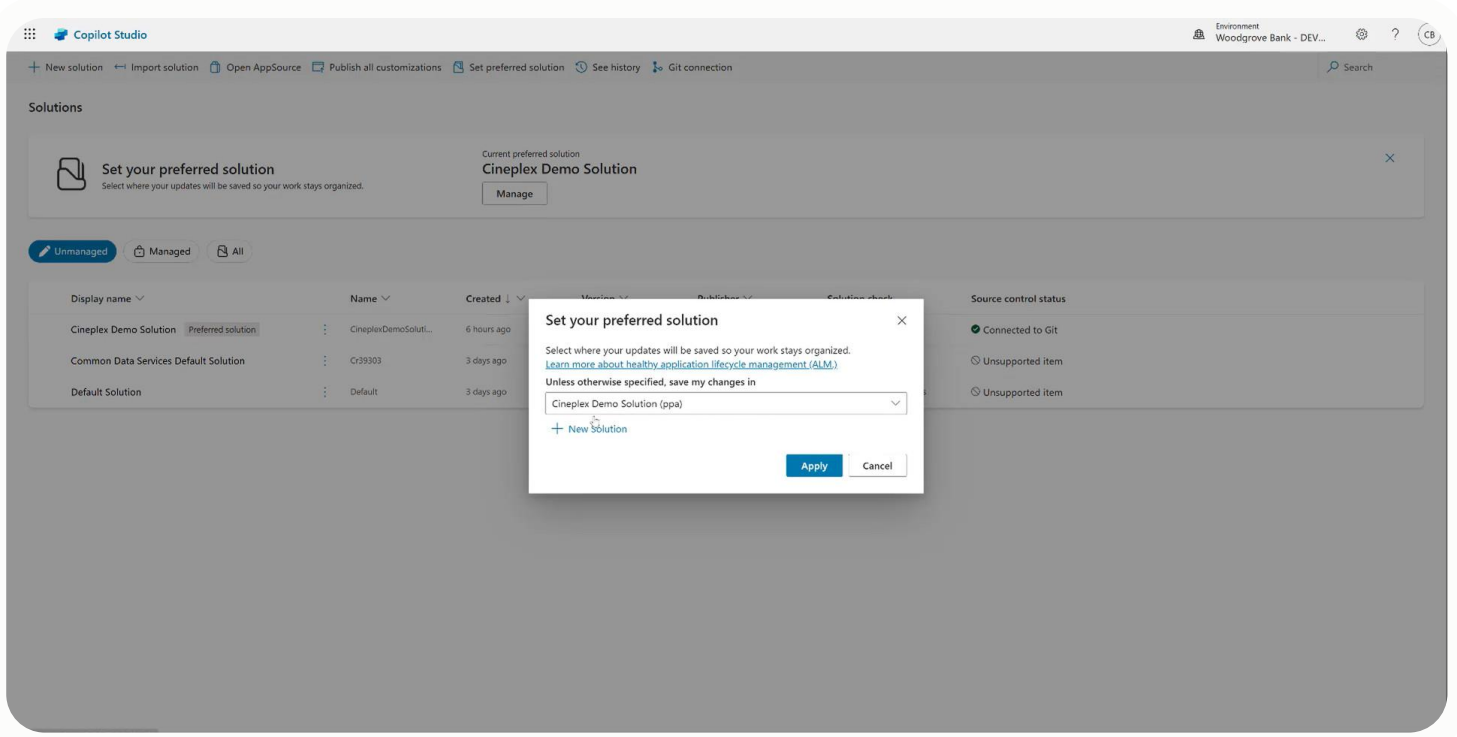
Best practice

Organizes customizations automatically

Works across all maker surfaces

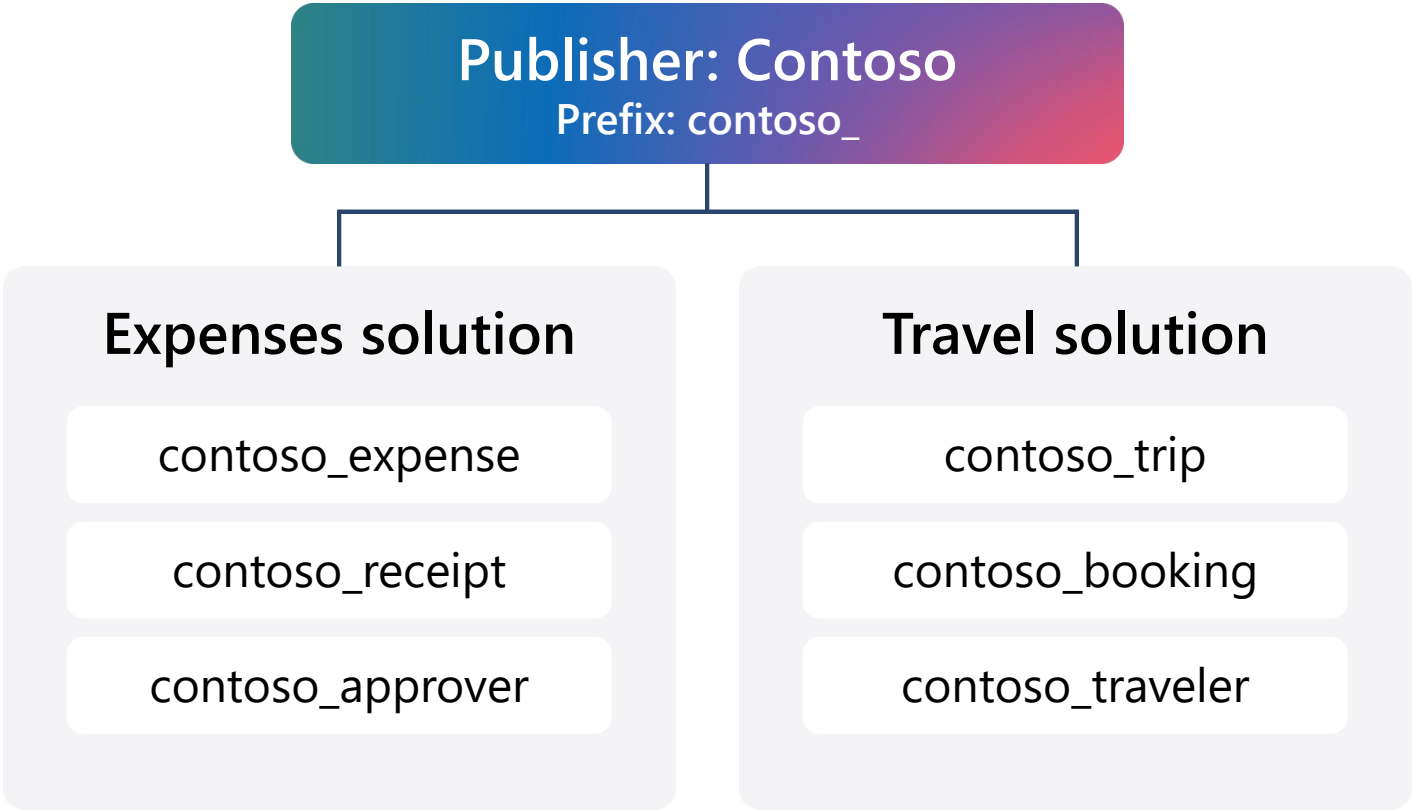
Ensures what you create uses the right solution and publisher

Don’t create/edit anything outside your solution context without a preferred solution.



What is a solution publisher?

Every solution has one. Use the same publisher for a project – or across projects.



Names it

Adds its prefix to every component you create.

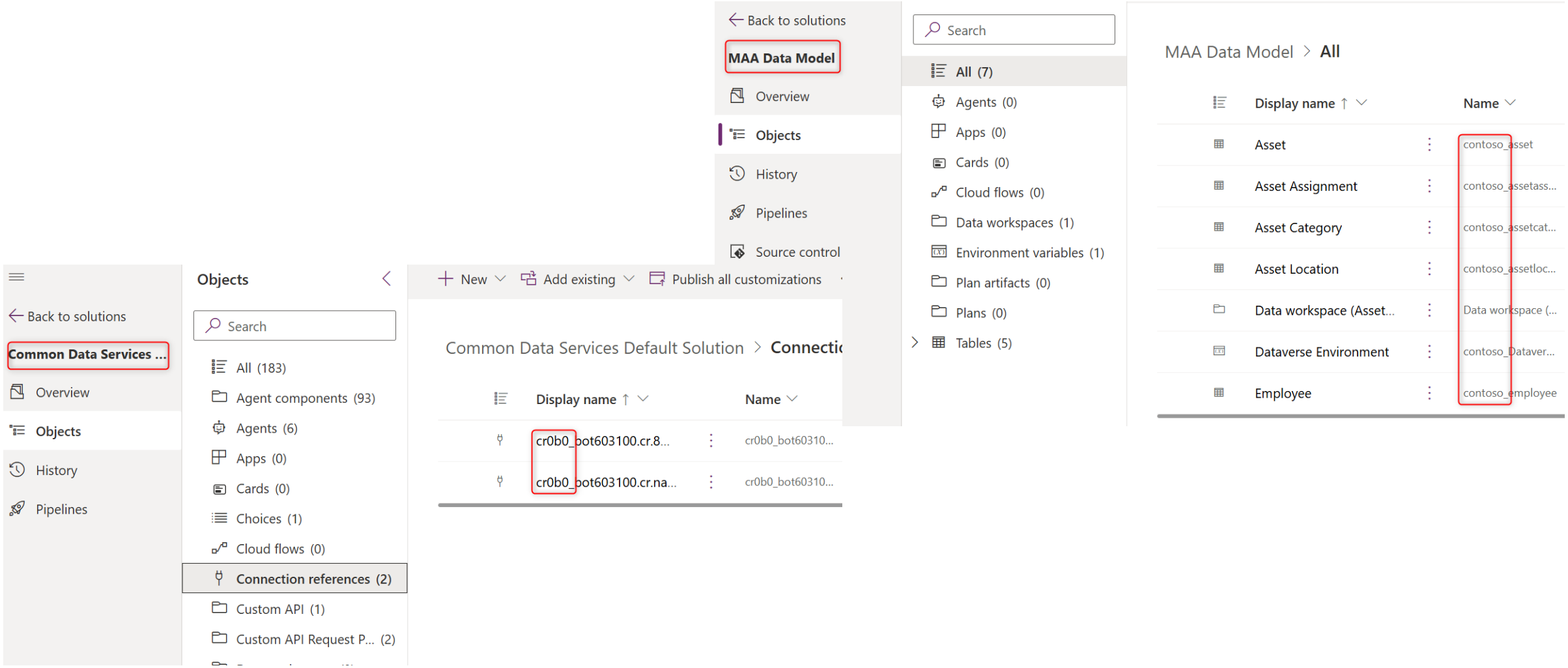
Owns it

Controls what other publishers can change.

Keeps it

Ownership can't move to another publisher. It's immutable.

Spot the bad and good



Unmanaged vs managed

Unmanaged

For development

Fully editable

Can be exported

Managed

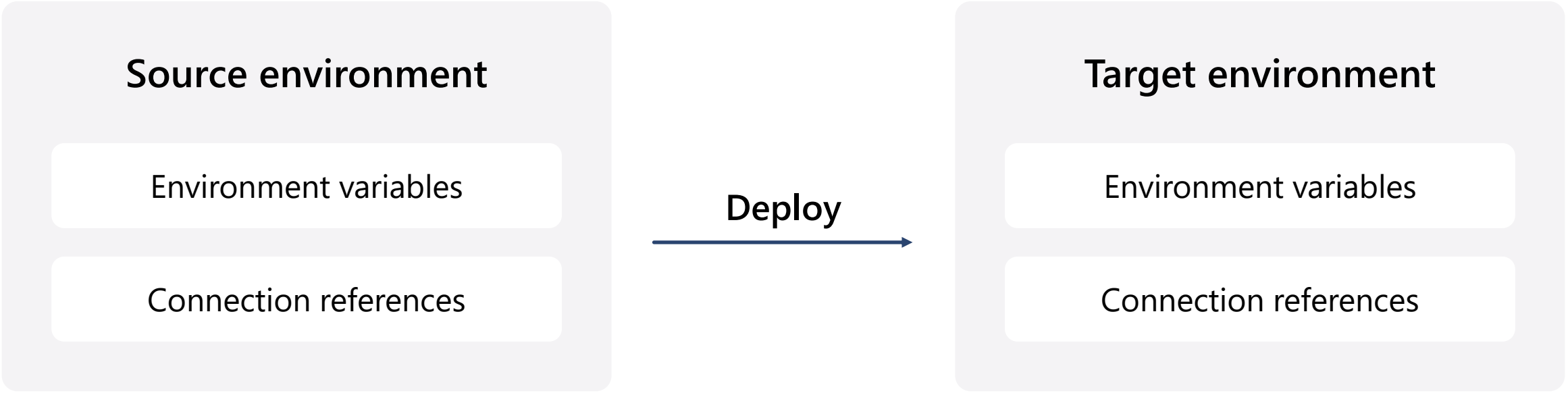
For deployment

Sealed, read-only

Can't be exported

Never deploy unmanaged to test or production

Environment variables & connection references



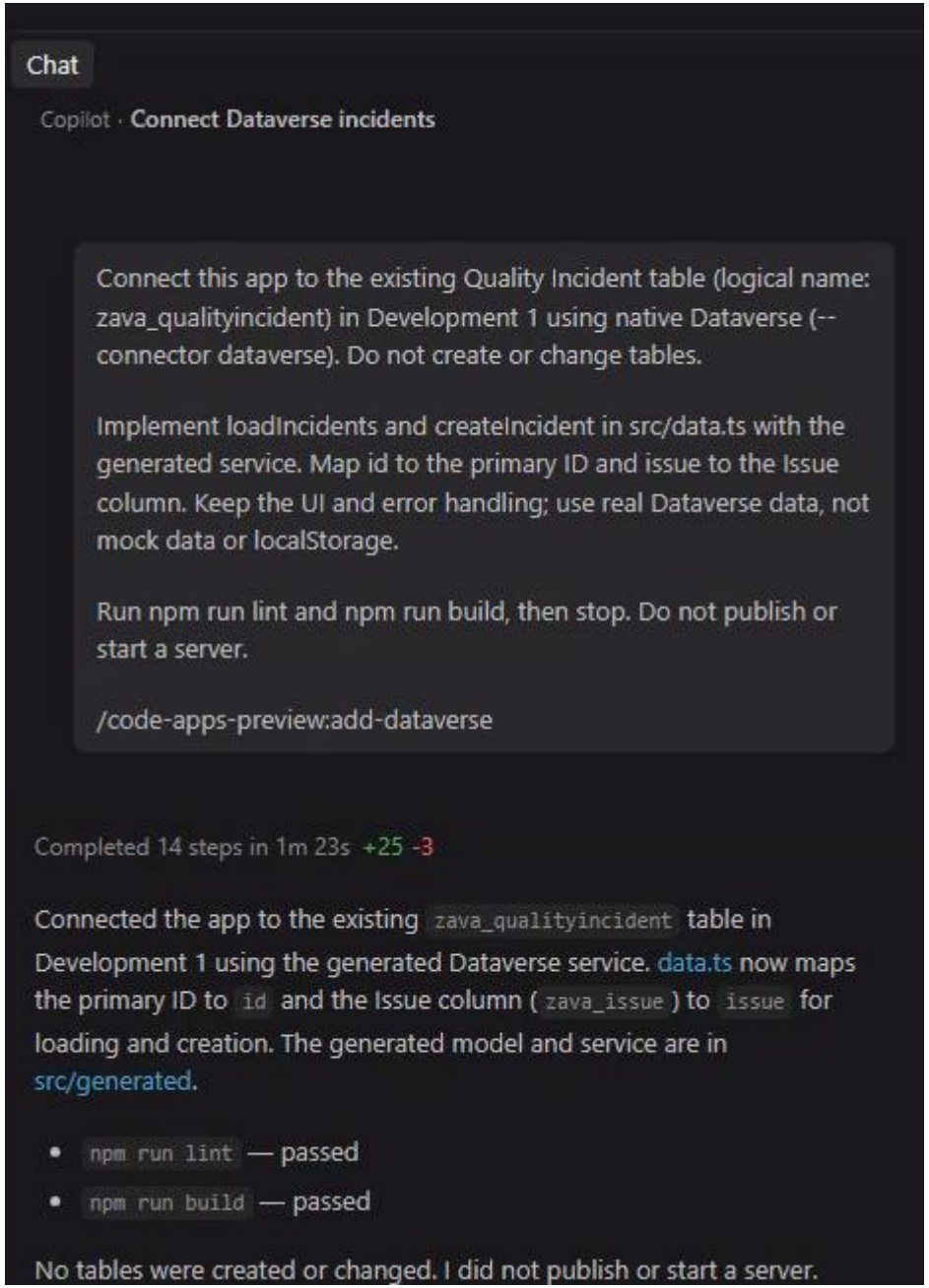
For things you need to change when deploying to another environment:

Connection references allow you to use different connections in dev vs test vs prod

Environment variables are a config value – e.g. app uses a different SharePoint site in test vs prod

Tools you'll use in this workshop

GitHub Copilot & VS Code



PAC CLI

```
PS C:\repo\ppcc26-alm\zava-quality-incident-tracker> git commit -m "Create Zava Quality Incident Tracker"
[master (root-commit) c62a83a] Create Zava Quality Incident Tracker
 22 files changed, 4911 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 .power/schemas/appschemas/dataSourcesInfo.ts
 create mode 100644 .power/schemas/dataverse/qualityincidents.Schema.json
 create mode 100644 README.md
 create mode 100644 eslint.config.js
 create mode 100644 index.html
 create mode 100644 package-lock.json
 create mode 100644 package.json
 create mode 100644 power.config.json
 create mode 100644 src/App.css
 create mode 100644 src/App.tsx
 create mode 100644 src/data.ts
 create mode 100644 src/generated/index.ts
 create mode 100644 src/generated/models/CommonModels.ts
 create mode 100644 src/generated/models/Zava_qualityincidentsModel.ts
 create mode 100644 src/generated/services/Zava_qualityincidentsService.ts
 create mode 100644 src/index.css
 create mode 100644 src/main.tsx
 create mode 100644 tsconfig.app.json
 create mode 100644 tsconfig.json
 create mode 100644 tsconfig.node.json
 create mode 100644 vite.config.ts
```

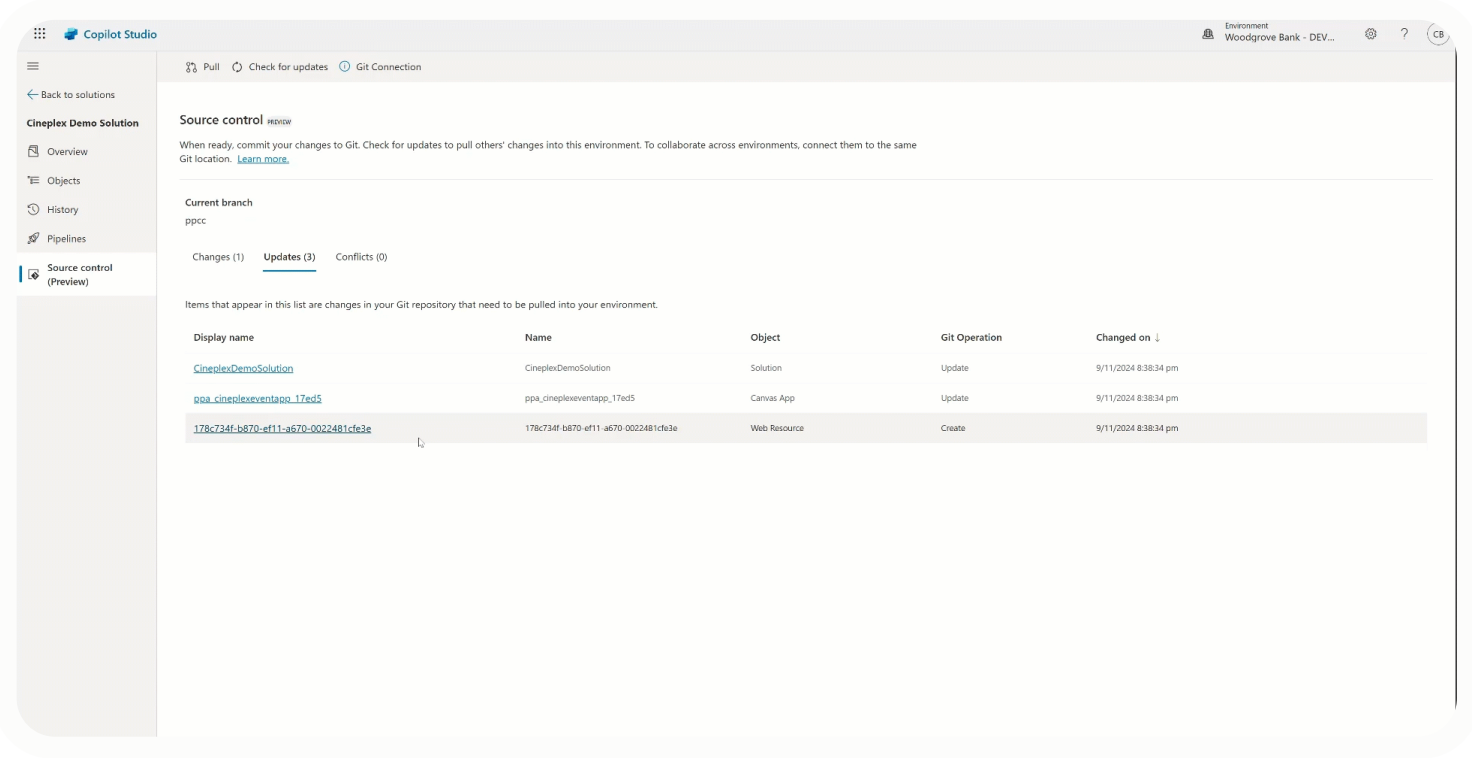
Native Source Code Integration

Sync assets with Git repos and other environments

YAML format is easier to read and merge

Commits delta changes only – not the entire solution

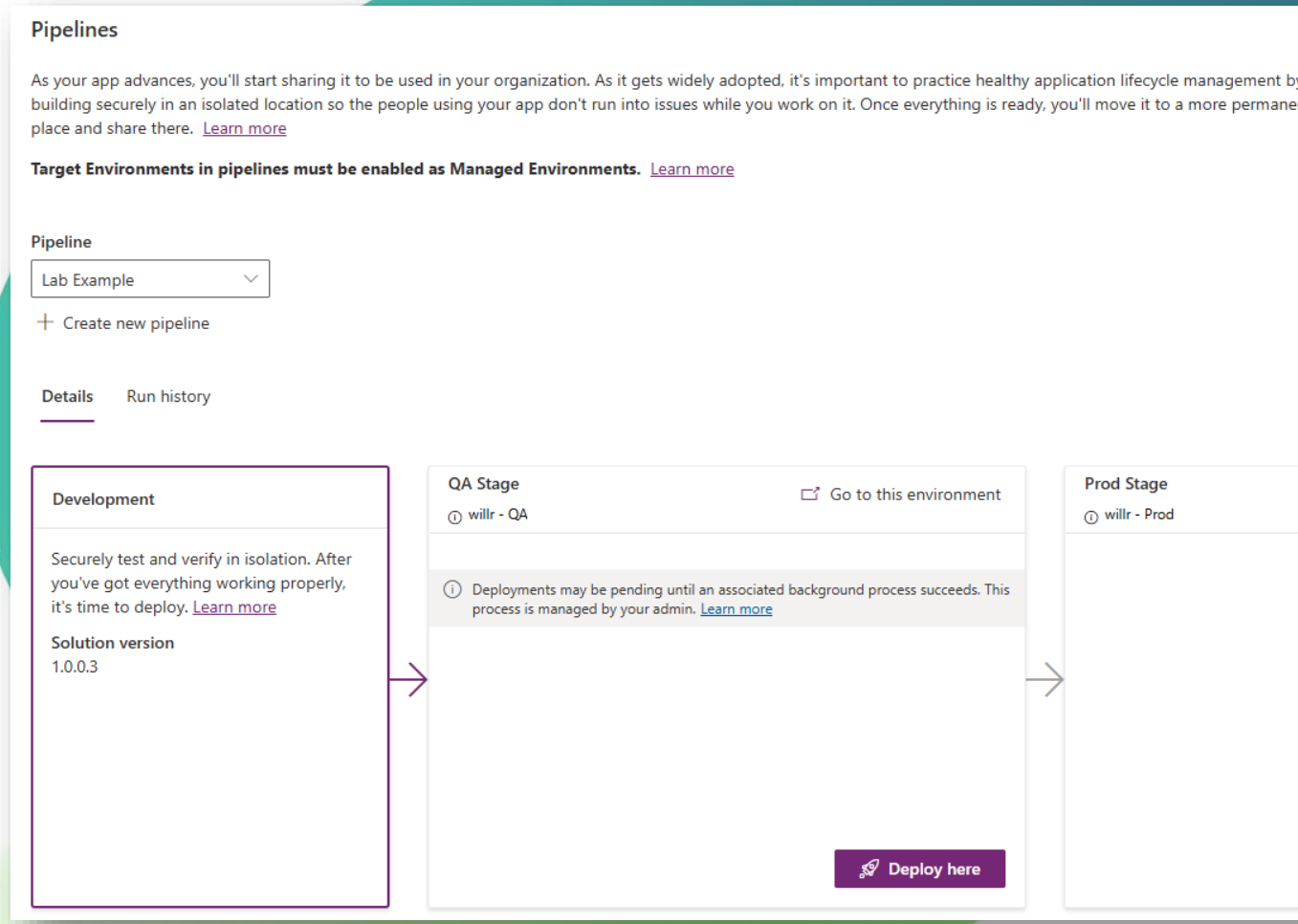
Azure DevOps (GA) and GitHub (preview)



aka.ms/SourceCodeIntegration

Pipelines in Power Platform

- Most used tool for automated deployments
- Self-service and easy to use
- Drives consistency for all makers, developers, and admins
- Fully **extensible**
- Suitable for 90%+ of projects



Microsoft Power Platform

Lab 1: Setup and Configure

Step 1: Get the labs

https://aka.ms/PPCC26-ALM

Step 2: Get a login

https://aka.ms/PPCCWorkshopAccounts

Agenda

Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

Microsoft Power Platform

Lab 2: Fusion and code-first development

Step 1: Get the labs

https://aka.ms/PPCC26-ALM

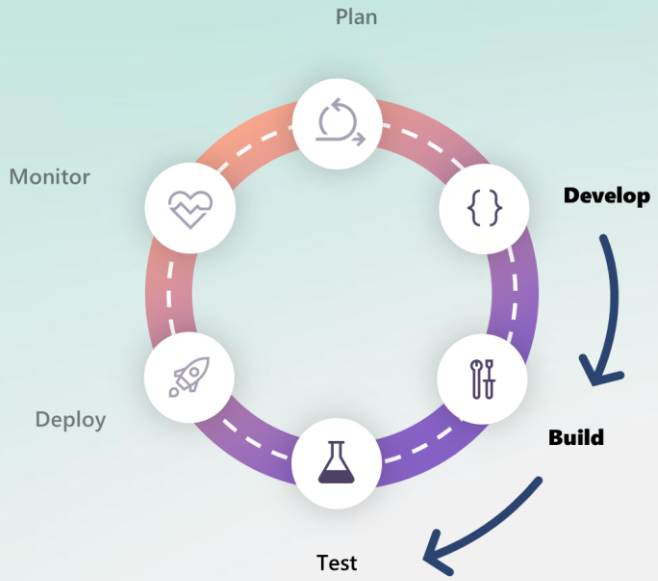
Step 2: Get a login

https://aka.ms/PPCCWorkshopAccounts

Agenda

Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

The inner loop



What is Source Code?

6. Assembly

Code:

```
MOV AX, 0       ; sum = 0
MOV CX, 1       ; i = 1
LOOP_START:
ADD AX, CX      ; sum += i
INC CX
CMP CX,
JL LOOP_
```

1. Copilot

Prompt:

"Calculate the sum of numbers from 1 to 10"

```
1  int sum = 0;
2  for(int i = 1; i <= 10; i++
3      sum += i;
```

2. Python

Code:

```
sum(range(1, 11))
```

3. C# (Object-Oriented)

Code:

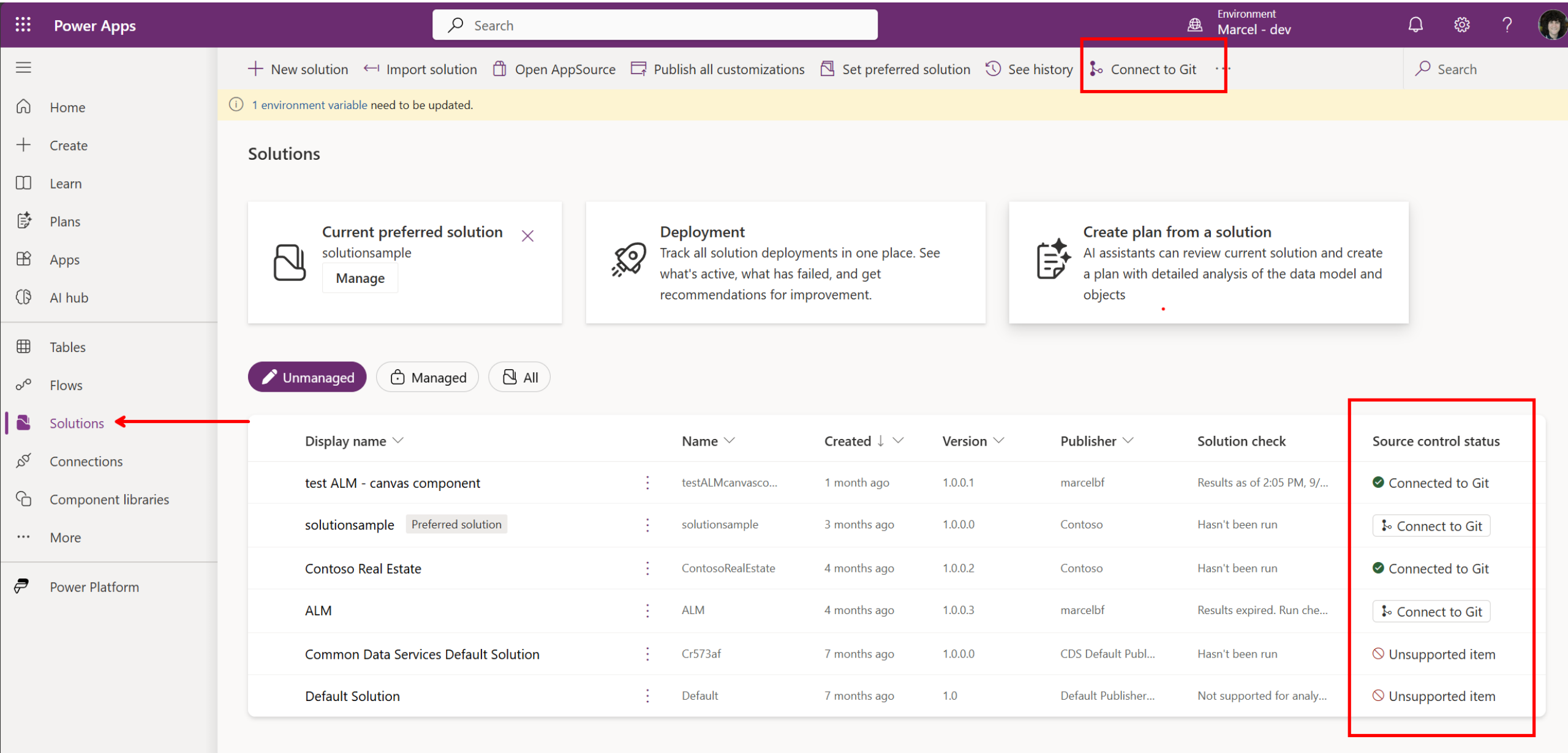
```
1  class Calculator {
2      public int Sum(int start, int end) {
3          int sum = 0;
4          for (int i = start; i <= end; i++)
5              sum += i;
6          return sum;
7      }
8  }
```

4. F# (Funct

Code:

```
let sum = [1..10] |> List.sum
```

Solution is the Source Code



Three representations. Different purposes.

The solution abstracts deployment. It does not replace every editable source.



Native solution Git supports specific component types; it is not a mirror of every solution-package file.
Developer source is not part of the solution. Keep their developer source in its own repository.

A solution package deploys. Solution source versions supported components. Developer source is where you code.

Build Zava Quality Operations

Help a quality team record incidents and know what to do next.

For example: a batch is missing its traceability record.

GUIDE THE RESPONSE

Zava Quality Guide

Ask how to handle the issue. Get guidance from the quality-triage skill.

CAPTURE THE ISSUE

Quality Incident Tracker

Use a code app with one Issue field to create and view quality incidents.

STORE THE RECORD

Quality Incident table

Keep incident records in Dataverse, separate for each environment.

Build in Development 1. Deploy the solution to Development 2.

Deploy metadata + built app assets. Developer source stays in Git; incident data stays in Dataverse.

LAB 2 - TASK 1

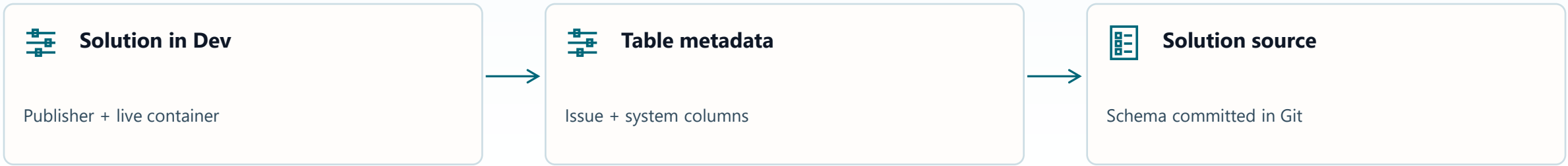
Create a solution

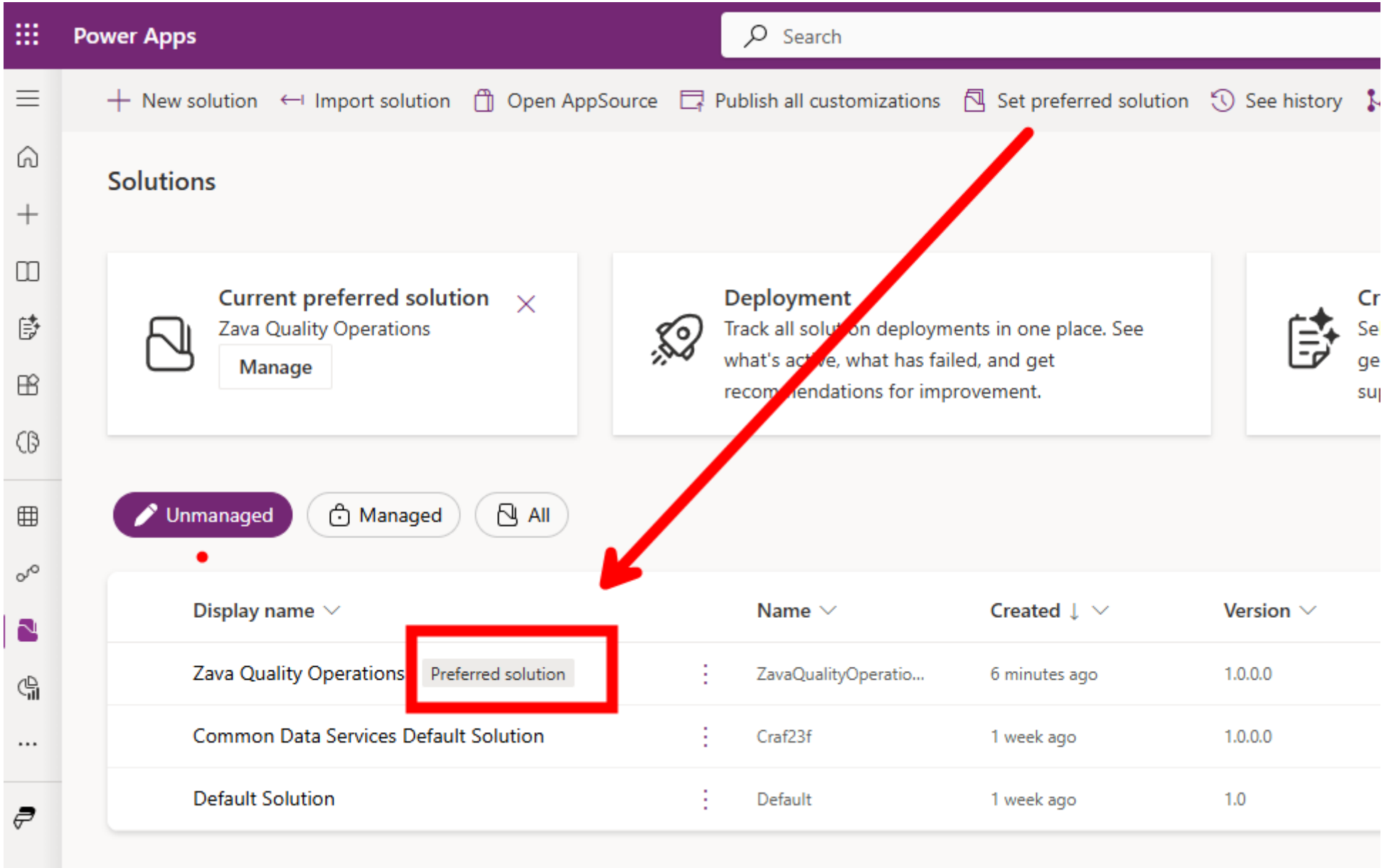
Estimated time: 10 minutes

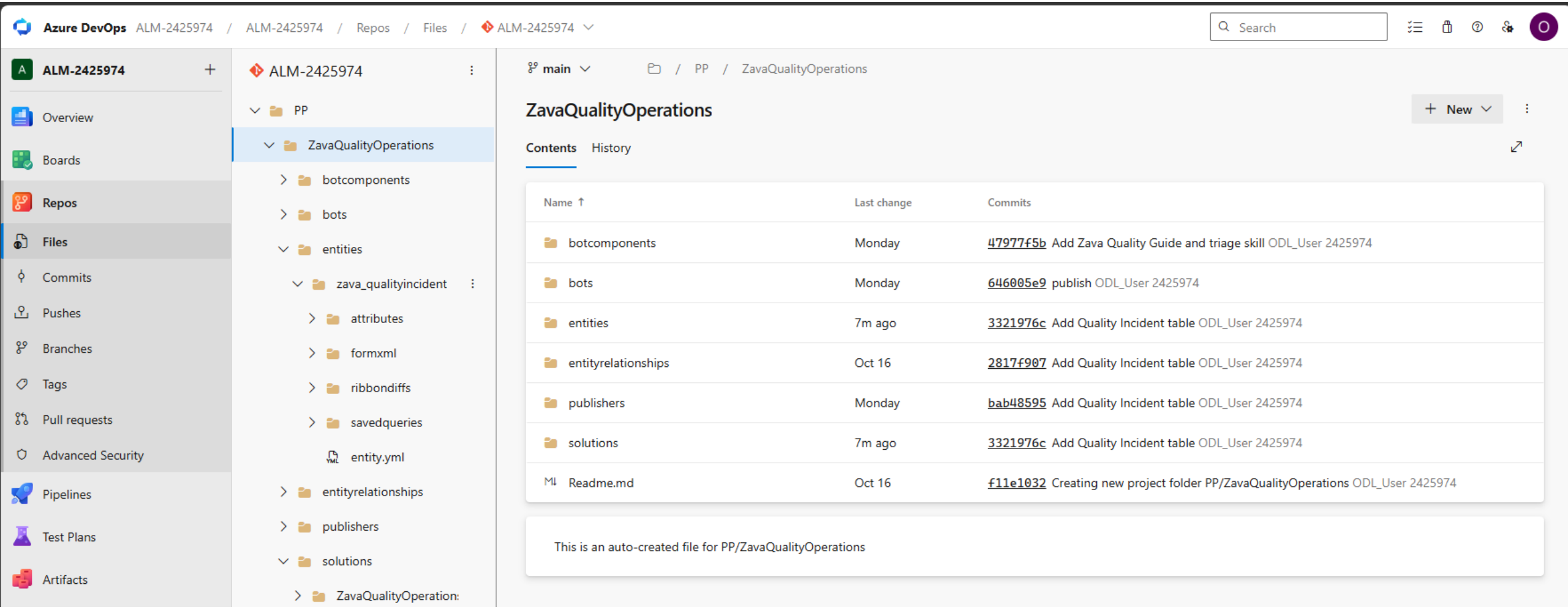
Maker portal + native solution Git

Expected outcome

- Publisher and preferred solution created in Dev.
- Quality Incident table has one business field: Issue.
- Solution connected to native Git integration.
- Table schema committed and inspected in the repository.




Power Apps
Search
New solution
Import solution
Open AppSource
Publish all customizations
Set preferred solution
See history
Solutions
Current preferred solution
Zava Quality Operations
Manage
Deployment
Track all solution deployments in one place. See what's active, what has failed, and get recommendations for improvement.
Unmanaged
Managed
All
Display name
Name
Created
Version
Zava Quality Operations
Preferred solution
ZavaQualityOperatio...
6 minutes ago
1.0.0.0
Common Data Services Default Solution
Craf23f
1 week ago
1.0.0.0
Default Solution
Default
1 week ago
1.0


Azure DevOps ALM-2425974 / ALM-2425974 / Repos / Files / ALM-2425974
Search
ALM-2425974
Overview
Boards
Repos
Files
Commits
Pushes
Branches
Tags
Pull requests
Advanced Security
Pipelines
Test Plans
Artifacts
ALM-2425974
PP
ZavaQualityOperations
botcomponents
bots
entities
zava_qualityincident
attributes
formxml
ribbondiffs
savedqueries
entity.yml
entityrelationships
publishers
solutions
ZavaQualityOperation
main / PP / ZavaQualityOperations
ZavaQualityOperations
New
Contents
History
Name
Last change
Commits
botcomponents Monday 47977f5b Add Zava Quality Guide and triage skill ODL_User 2425974
bots Monday 646005e9 publish ODL_User 2425974
entities 7m ago 3321976c Add Quality Incident table ODL_User 2425974
entityrelationships Oct 16 2817f907 Add Quality Incident table ODL_User 2425974
publishers Monday bab48595 Add Quality Incident table ODL_User 2425974
solutions 7m ago 3321976c Add Quality Incident table ODL_User 2425974
Readme.md Oct 16 f11e1032 Creating new project folder PP/ZavaQualityOperations ODL_User 2425974
This is an auto-created file for PP/ZavaQualityOperations

LAB 2 - TASK 2

Create an agent

Estimated time: 10 minutes

Copilot Studio / Microsoft Copilot harness

Microsoft Power Platform

Coffee Break 15 min

Agenda

Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

LAB 2 - TASK 2

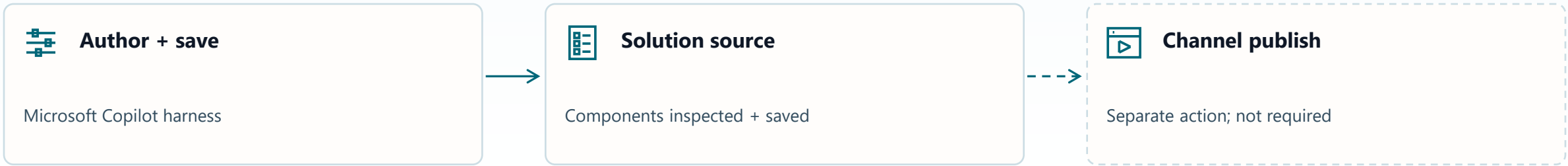
Create an agent

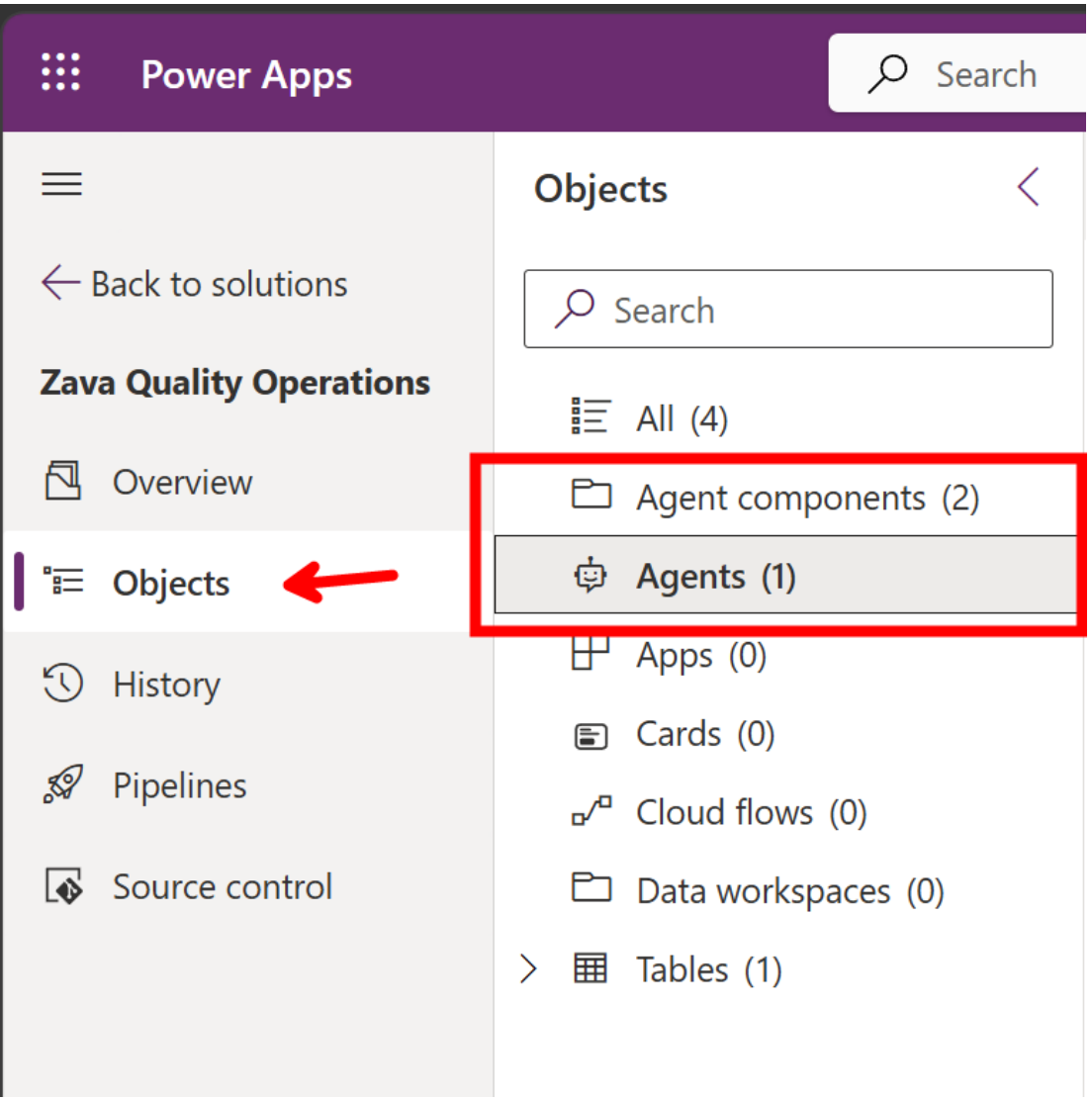
Estimated time: 10 minutes

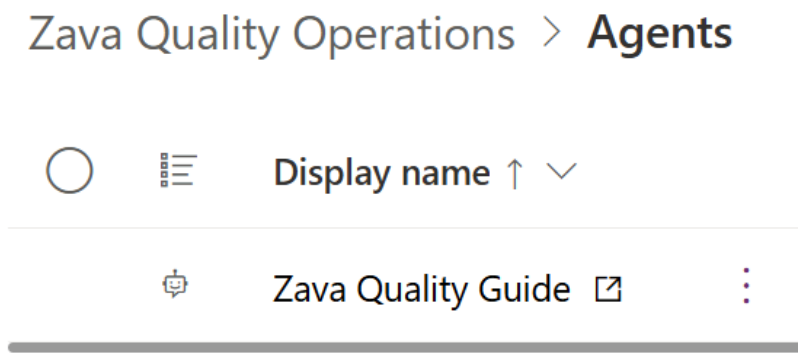
Copilot Studio / Microsoft Copilot harness

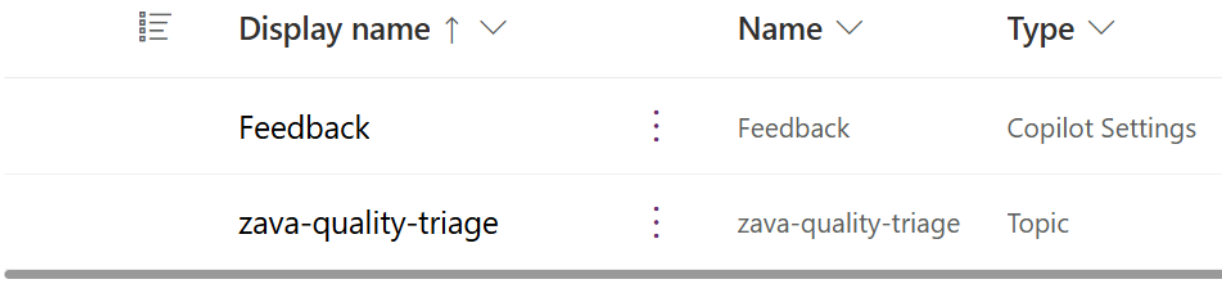
Expected outcome

- Agent created with the Microsoft Copilot harness.
- Agent and related components browsed in the preferred solution.
- Supported configuration committed as solution source.
- Creating / saving is not publishing the agent to a channel.




Power Apps
Search
Back to solutions
Zava Quality Operations
Overview
Objects
History
Pipelines
Source control
Objects
Search
All (4)
Agent components (2)
Agents (1)
Apps (0)
Cards (0)
Cloud flows (0)
Data workspaces (0)
Tables (1)


Zava Quality Operations > Agents
Display name
Zava Quality Guide


Zava Quality Operations > Agent components
Display name
Name
Type
Feedback
Feedback
Copilot Settings
zava-quality-triage
zava-quality-triage
Topic

ALM-2425974

Overview
Boards
Repos
Files
Commits
Pushes
Branches
Tags
Pull requests
Advanced Security
Pipelines
Test Plans
Artifacts

Project settings

ALM-2425974

PP
ZavaQualityOperation
botcomponents
bots
entities
entityrelationships
publishers
solutions
ZavaQualityOper
Readme.md
ZavaQualityOperation
ZavaQualityOperation
ZavaQualityOperation
ZavaQualityOperation
ZavaQualityOperation
README.md

zava-quality-inspection-guide.md

Edit

Contents Preview History Compare Blame

```
# ZavaCore Quality Inspection Guide

This synthetic guide exists only for the PPCC ALM workshop. It is not a real Zava or Microsoft manufacturing policy.

## Purpose

Use this guide to classify and escalate quality issues found during inspection of ZavaCore products.

## Required incident information

Record:

- Batch ID
- ZavaCore product
- Date and time observed
- Short description of the issue
- Severity
- Current status
- Assigned owner

Do not include customer personal data, employee personal data, credentials, or secrets.

## Severity 1: Observation

Use **Observation** when:

- The issue is cosmetic.
- Product performance and safety are unaffected.
- Packaging or labeling has a minor defect but remains readable and traceable.

Action:

1. Record the observation.
2. Continue inspection.
3. Assign an owner before the end of the shift.

## Severity 2: Quality hold

Use **Quality hold** when:

- A required measurement is outside its approved target
```

LAB 2 - TASK 3

Create a code app locally

Estimated time: 15 minutes

Prepared project + one bounded Dataverse-binding request

Expected outcome

- Developer source created on the local machine and committed in Git.
- One Issue input reads and creates real Dataverse rows.
- Code app not in the solution yet; developer project stays outside.
- Business data stays in Dataverse, not in the solution package.

Developer source

Local project + its own Git

Solution in Dev

No code app yet

Business data

Real rows, outside package

Chat

Copilot · **Connect Dataverse incidents**

Connect this app to the existing Quality Incident table (logical name: zava_qualityincident) in Development 1 using native Dataverse (--connector dataverse). Do not create or change tables.

Implement loadIncidents and createIncident in src/data.ts with the generated service. Map id to the primary ID and issue to the Issue column. Keep the UI and error handling; use real Dataverse data, not mock data or localStorage.

Run npm run lint and npm run build, then stop. Do not publish or start a server.

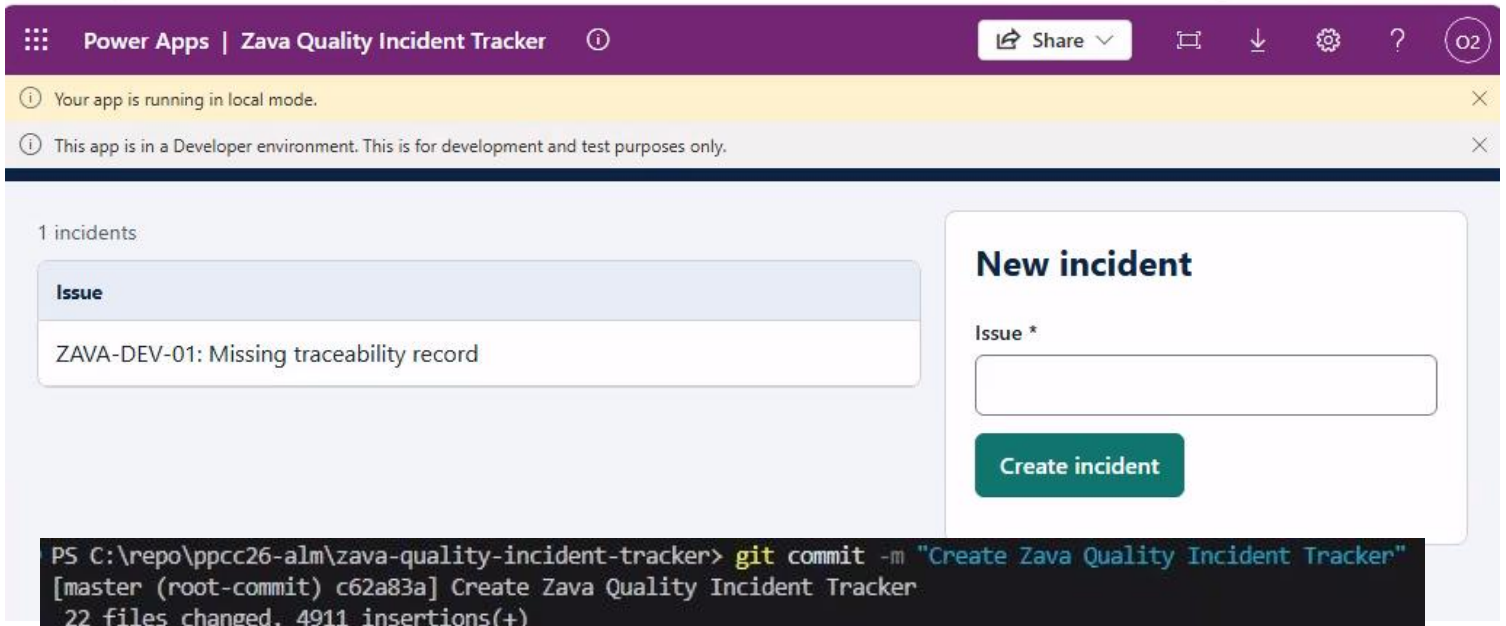
/code-apps-preview:add-dataverse

Completed 14 steps in 1m 23s +25 -3

Connected the app to the existing `zava_qualityincident` table in Development 1 using the generated Dataverse service. data.ts now maps the primary ID to `id` and the Issue column (`zava_issue`) to `issue` for loading and creation. The generated model and service are in src/generated.

- `npm run lint` — passed
- `npm run build` — passed

No tables were created or changed. I did not publish or start a server.




```
PS C:\repo\ppcc26-alm\zava-quality-incident-tracker> git commit -m "Create Zava Quality Incident Tracker"
[master (root-commit) c62a83a] Create Zava Quality Incident Tracker
 22 files changed, 4911 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 .power/schemas/appschemas/dataSourcesInfo.ts
 create mode 100644 .power/schemas/dataverse/qualityincidents.Schema.json
 create mode 100644 README.md
 create mode 100644 eslint.config.js
 create mode 100644 index.html
 create mode 100644 package-lock.json
 create mode 100644 package.json
 create mode 100644 power.config.json
 create mode 100644 src/App.css
 create mode 100644 src/App.tsx
 create mode 100644 src/data.ts
 create mode 100644 src/generated/index.ts
 create mode 100644 src/generated/models/CommonModels.ts
 create mode 100644 src/generated/models/Zava_qualityincidentsModel.ts
 create mode 100644 src/generated/services/Zava_qualityincidentsService.ts
 create mode 100644 src/index.css
 create mode 100644 src/main.tsx
 create mode 100644 tsconfig.app.json
 create mode 100644 tsconfig.json
 create mode 100644 tsconfig.node.json
 create mode 100644 vite.config.ts
```

LAB 2 - TASK 4

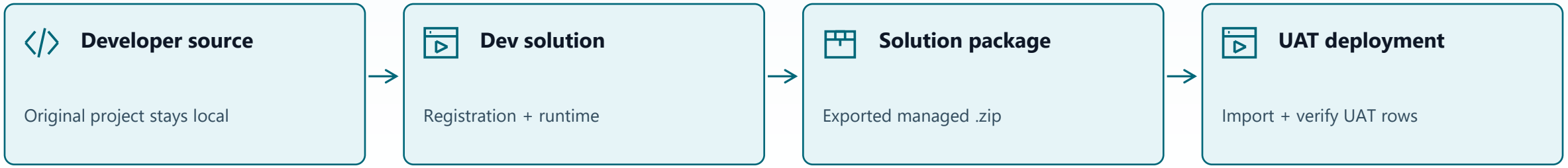
Add the app. Deploy the solution.

Estimated time: 10 minutes

Publish app to Dev > managed solution package > UAT

Expected outcome

- Dev solution contains app registration metadata + runtime assets.
- Original developer source is not included or reliably recoverable.
- Managed solution package exported and imported into Dev 2.
- You understand the solution includes the runtime, not the Source Code (React / TypeScript)



Build

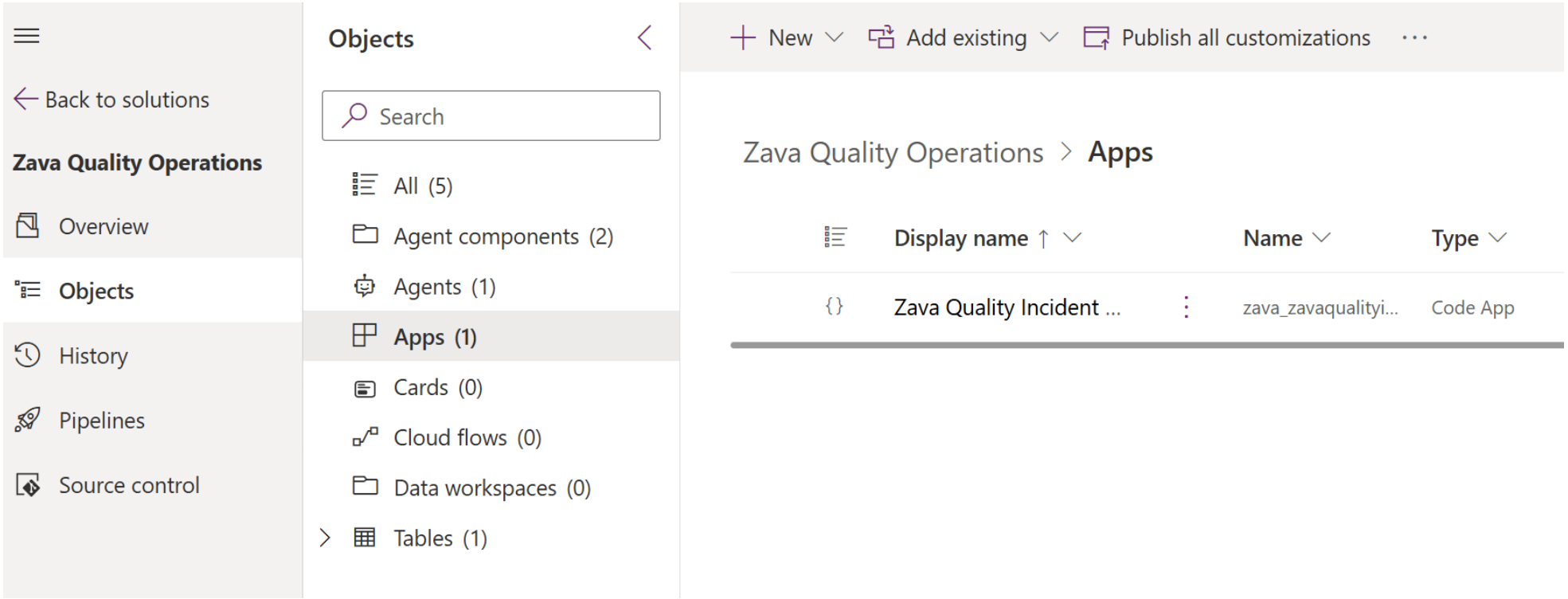
```
PS C:\repo\ppcc26-alm\zava-quality-incident-tracker> npm run build

> zava-quality-incident-tracker@1.0.0 build
> tsc -b && vite build

vite v7.3.7 building client environment for production...
✓ 74 modules transformed.
dist/index.html                   0.42 kB │ gzip:  0.28 kB
dist/assets/index-Cat5koa4.css    2.98 kB │ gzip:  1.17 kB
dist/assets/index-CnVSbfTb.js   279.59 kB │ gzip: 84.04 kB
✓ built in 2.23s
```

Add to the preferred solution

```
PS C:\repo\ppcc26-alm\zava-quality-incident-tracker> npx --no-install pa app push
Adding the app to your preferred solution.
App pushed successfully. You can play your app at https://apps.powerapps.com/play/e/b6b224b3-3874-ea78
81dc-7c43ddaae5c1&hint=8d638114-1909-43fd-9265-22e57fb65f78&sourcetime=1792822807583
Hint: Run 'pa app run' to test locally.
```



Three representations. Different purposes.

The solution abstracts deployment. It does not replace every editable source.



Native solution Git supports specific component types; it is not a mirror of every solution-package file.
Developer source is not part of the solution. Keep their developer source in its own repository.

A solution package deploys. Solution source versions supported components. Developer source is where you code.

BRANCHES + ENVIRONMENTS

Working as a Team in multiple environments

Branching

Main branch is protected

KEY IDEA

Main branch represents production and can only be merged via PR after deployment

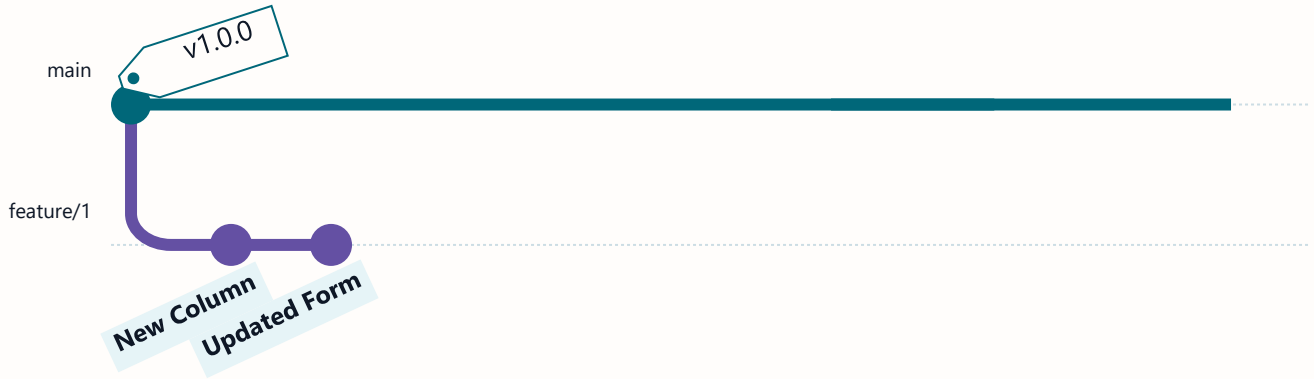
Tags represent the released version



Feature 1 branch and environment

KEY IDEA

Feature 1 Development starts on a branch of main

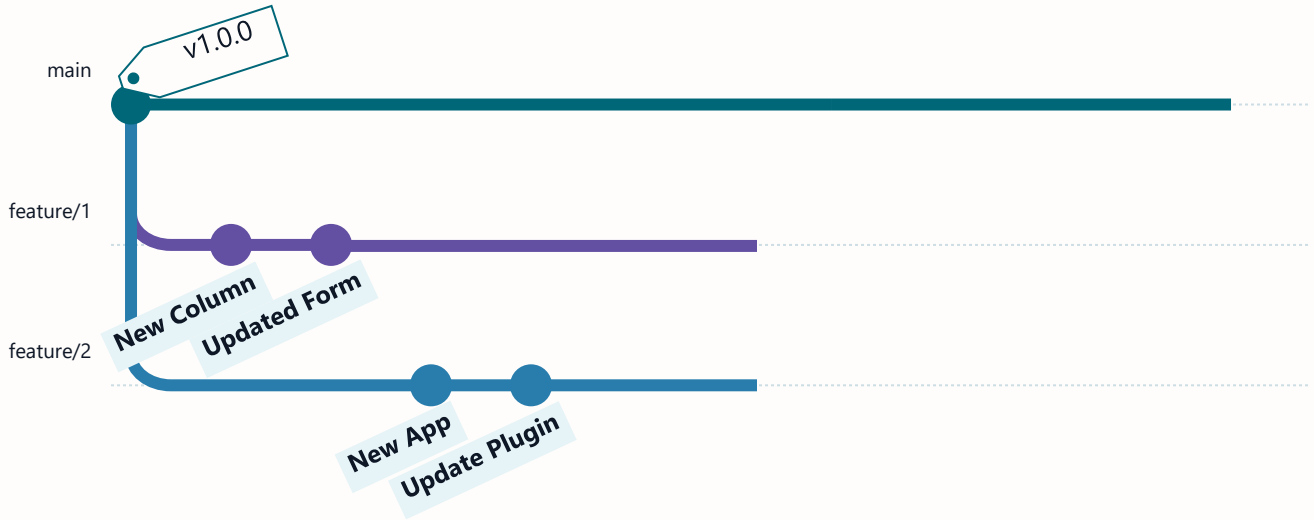


Feature 2 Branch and Environment

KEY IDEA

Feature 2 will likely be released after feature 1

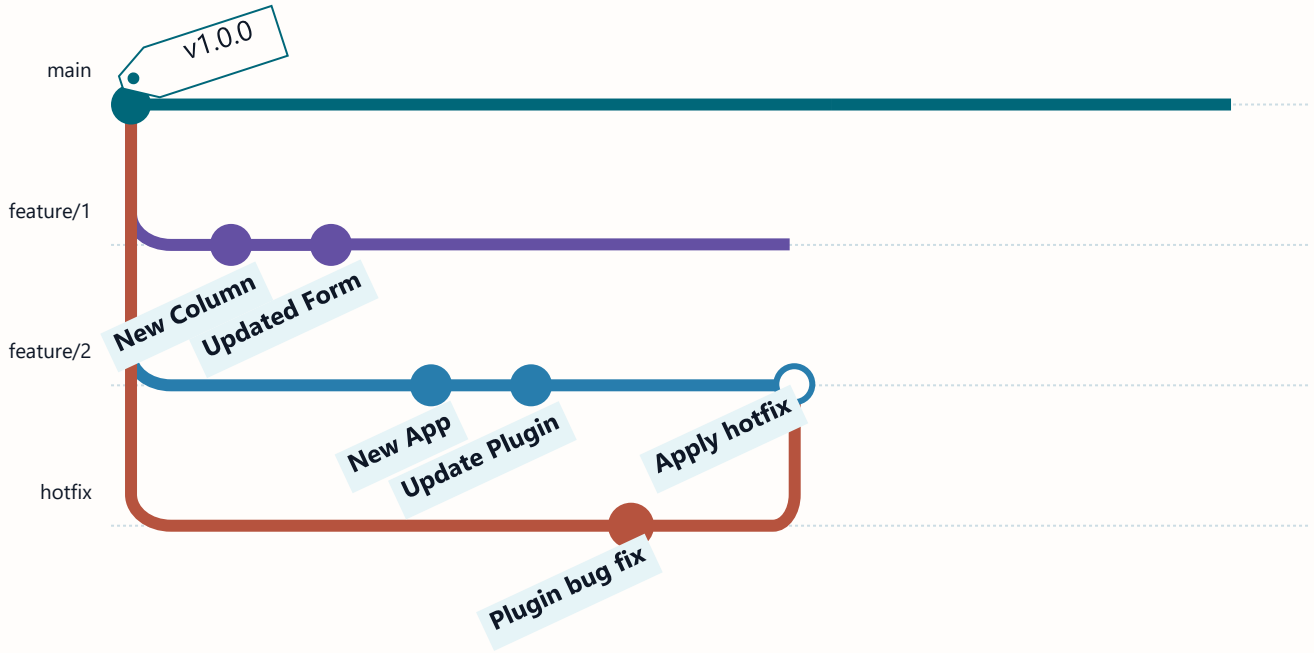
Feature 2 development starts in parallel on a branch of main in a **NEW ENVIRONMENT**



Hotfix created

KEY IDEA

A bug is discovered in production, and a hotfix is created in a branch of main in a **NEW ENVIRONMENT**

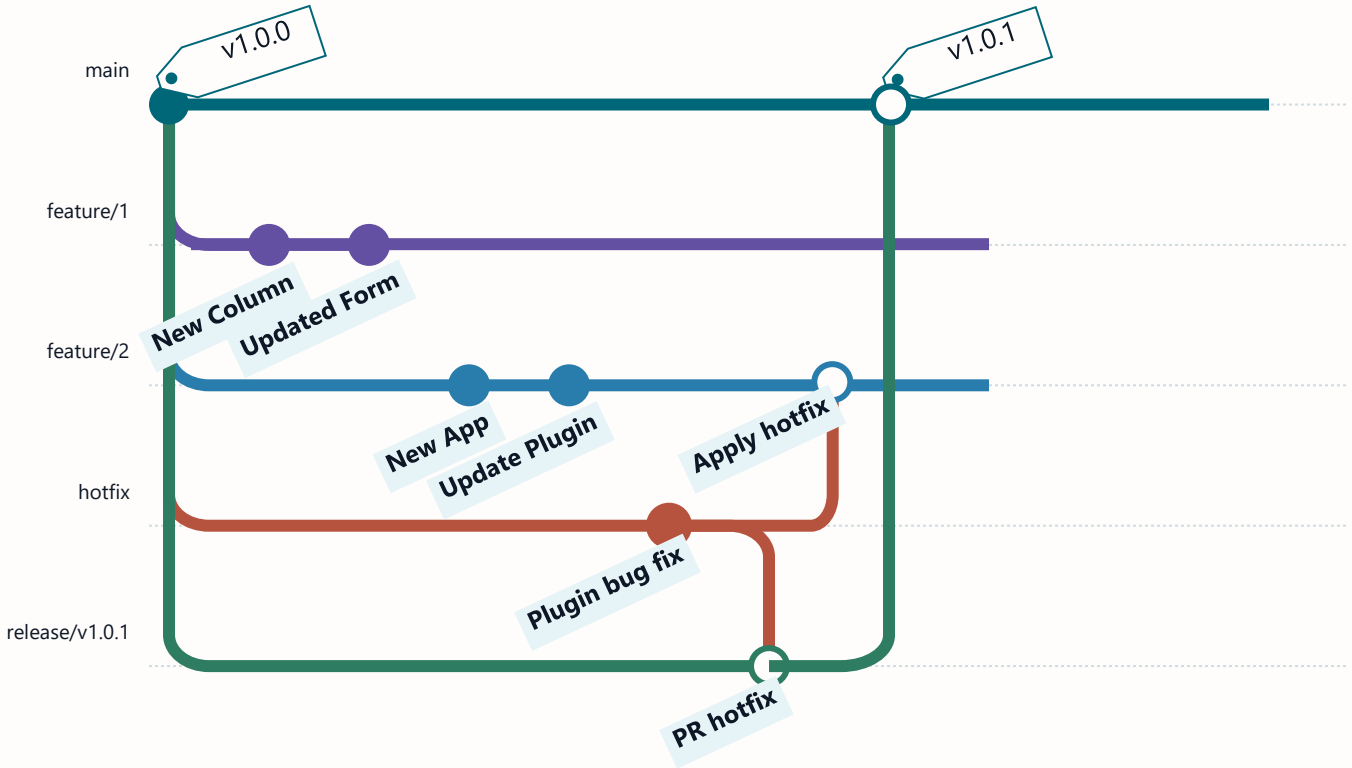


Hotfix Released and merged

KEY IDEA

A Release branch and NEW ENVIRONMENT is created, and the hotfix is merged in, and deployed. The hot fix is merged back into feature 2 because it effects the same code.

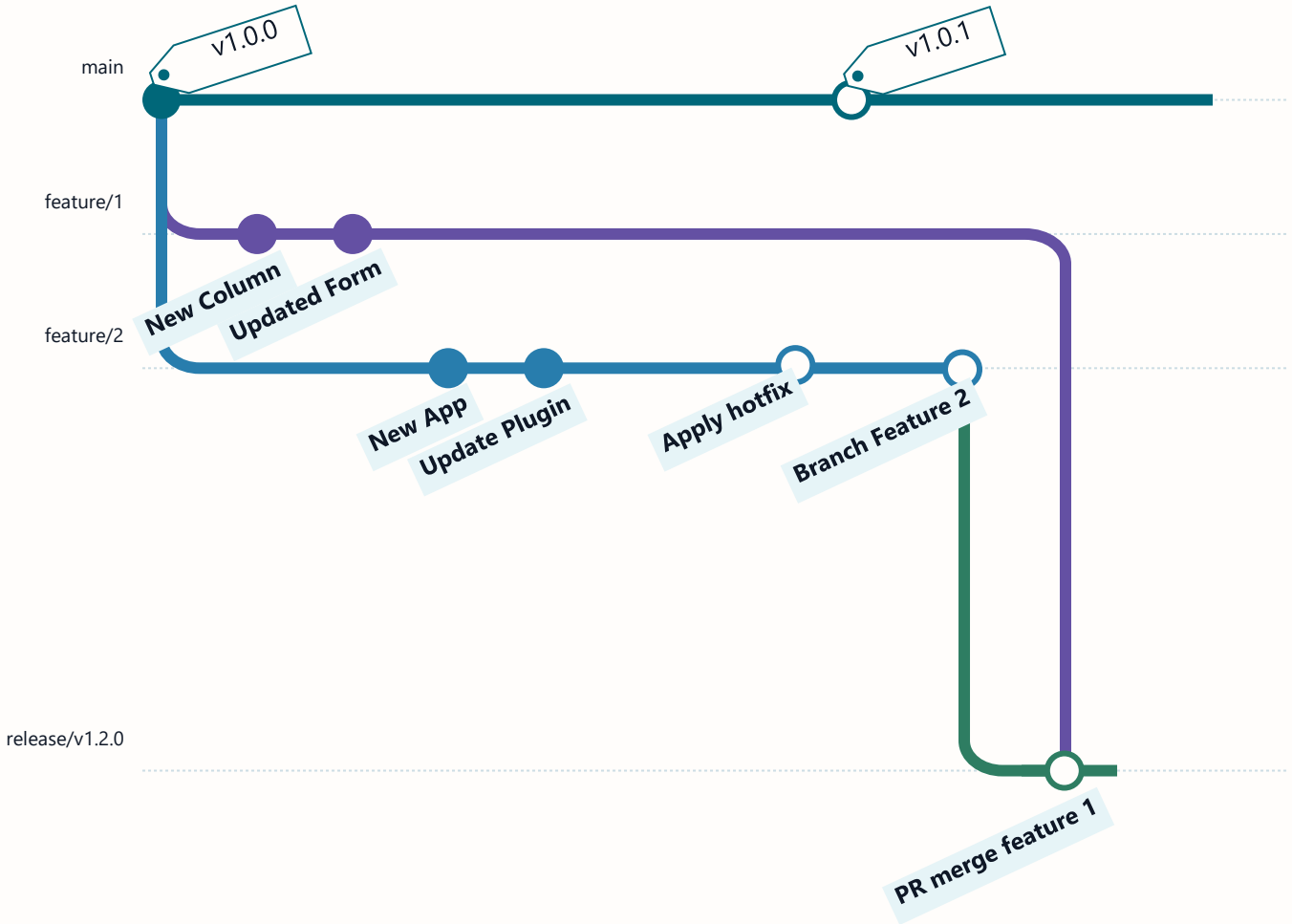
Version 1.0.1 hotfix is released and merged to main



Branch for release

KEY IDEA

Version 2 branch is created from feature 2 that contains the hotfix, and deployed to a new environment

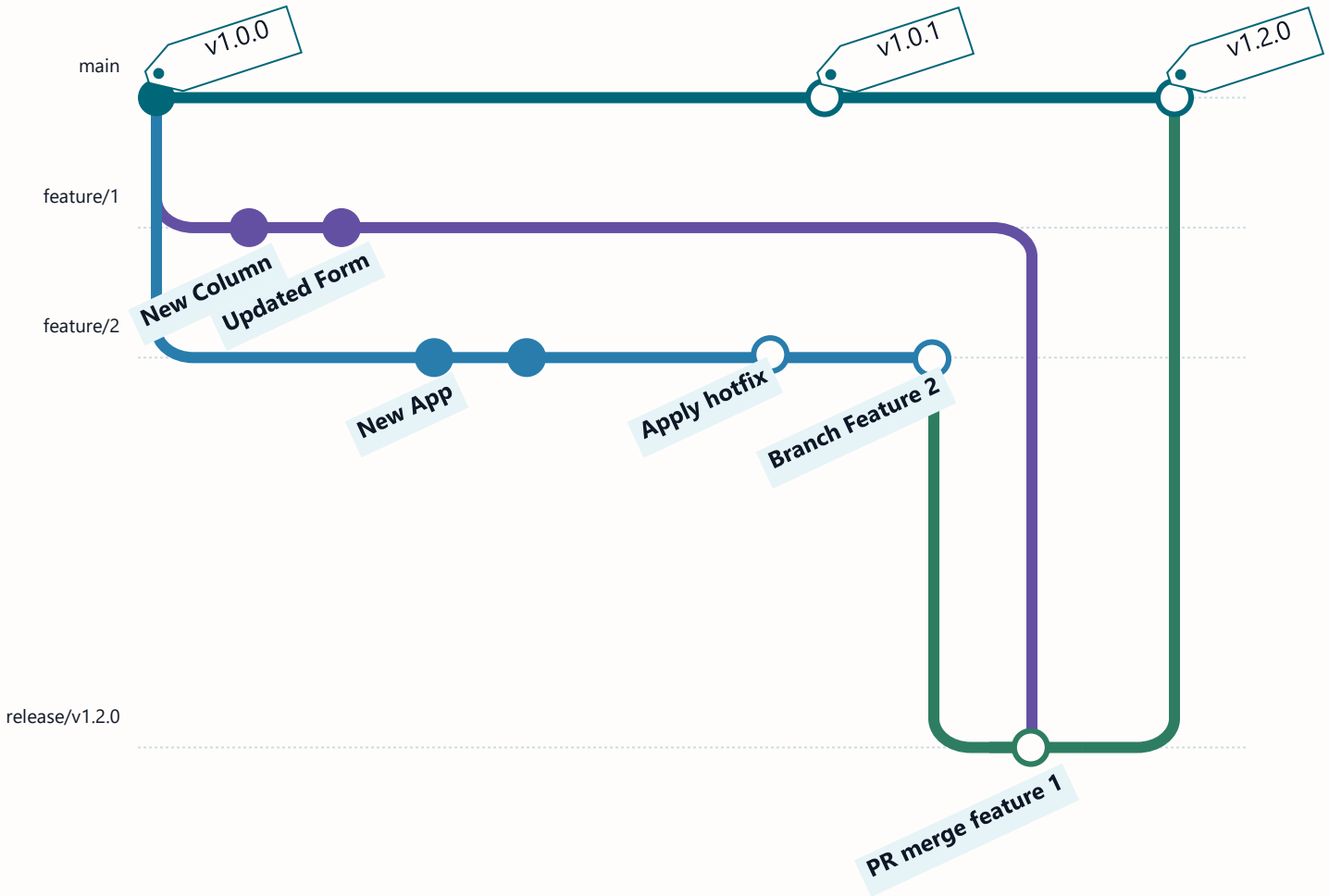


Deploy V2

KEY IDEA

Feature 1 is merged in. No merge conflicts should exist. If they do, the **changes must be back-ported** to feature 1.

Version 2 is deployed.



Microsoft Power Platform

Lab 3: Deploy

Agenda

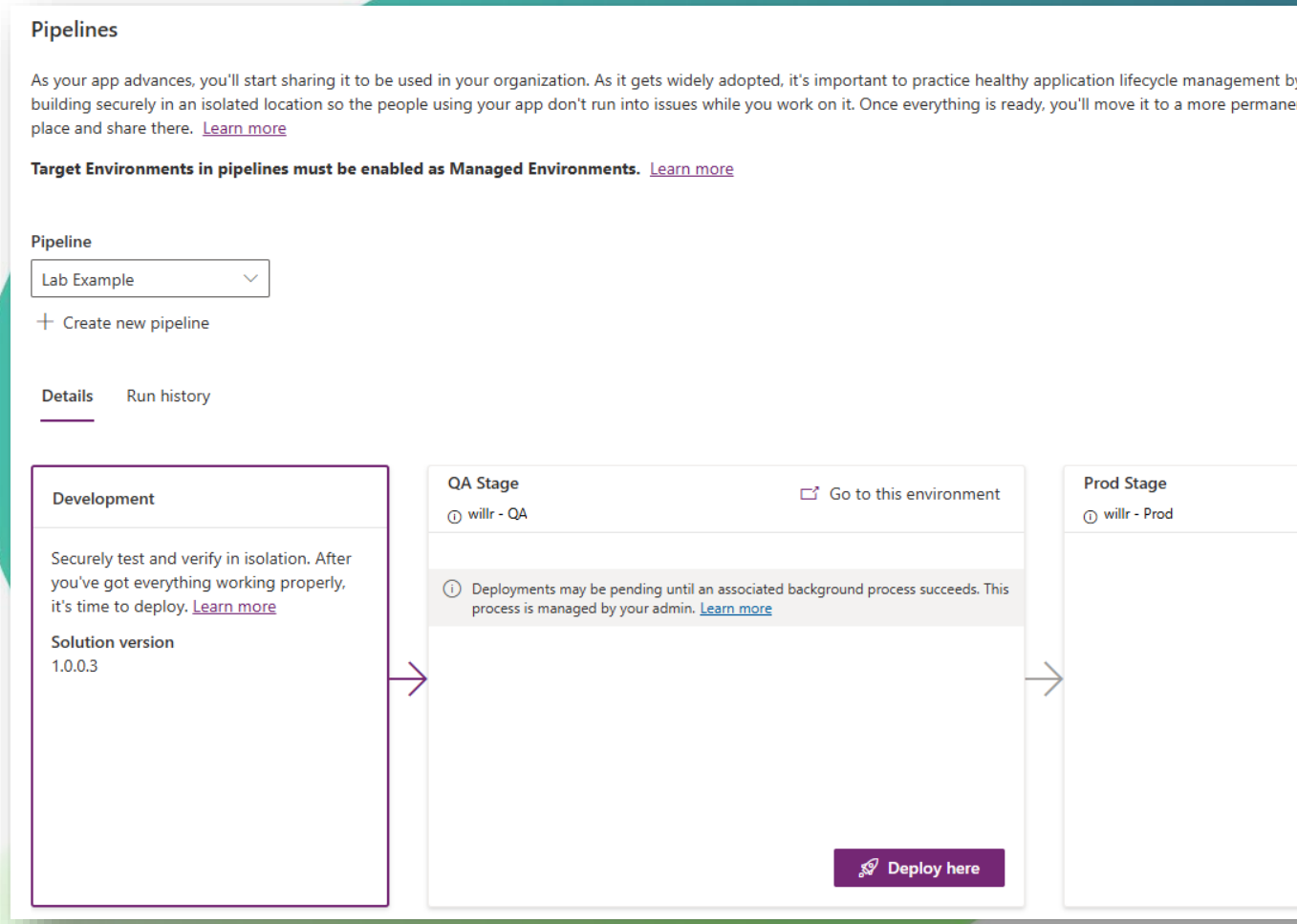
Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

Lab Overview

- Introduction to pipelines in Power Platform
- Create a Platform host pipeline
- Create a Custom host pipeline
- Extend the Custom host pipeline
- Redeploy using pipeline

Power Platform Pipelines

Pipelines in Power Platform offer an OOB ALM experience for Power Platform by bringing ALM automation and CI/CD capabilities into the service for all makers, admins, and developers.



Why use pipelines in Power Platform?

- Pipelines standardize solution movement from dev to test to production, reducing manual errors and ensuring predictable outcomes
- Common steps like export, import, and publish are automated, saving time and reducing operational overhead.
- They enforce environment security and approval workflows, aligning with organizational compliance requirements.
- Deployment logs and approval history provide transparency for troubleshooting and meeting audit requirements.
- Automatically stores managed and unmanaged versions of your solution for easy rollbacks– if needed.
- Included with Managed Environments

There are TWO types of pipelines

Platform Host

- Out of the box tenant-wide pipelines hosted under a “platform host” configuration
- These pipelines are configured by makers for **personal** use. Personal pipelines cannot be extended, nor can they be shared with other makers in the tenant

Custom Host

- Hosted on a “custom host” env installed and managed by admins
- Enable greater **control** as they are centrally governed by admins and shared with makers for use.
- Provide greater **visibility**, like audit where a certain solution is deployed and who initiated the request.
- Custom host pipelines also retain **backups** of solution artifacts in both Managed and Unmanaged forms.

Microsoft Power Platform

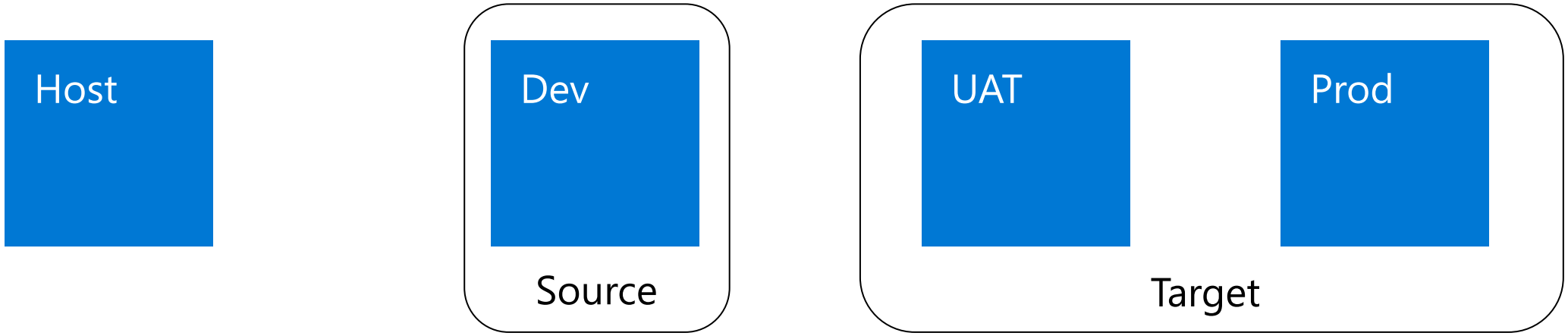
Create a Platform Host Personal Pipeline

Lab 3 – Task 1

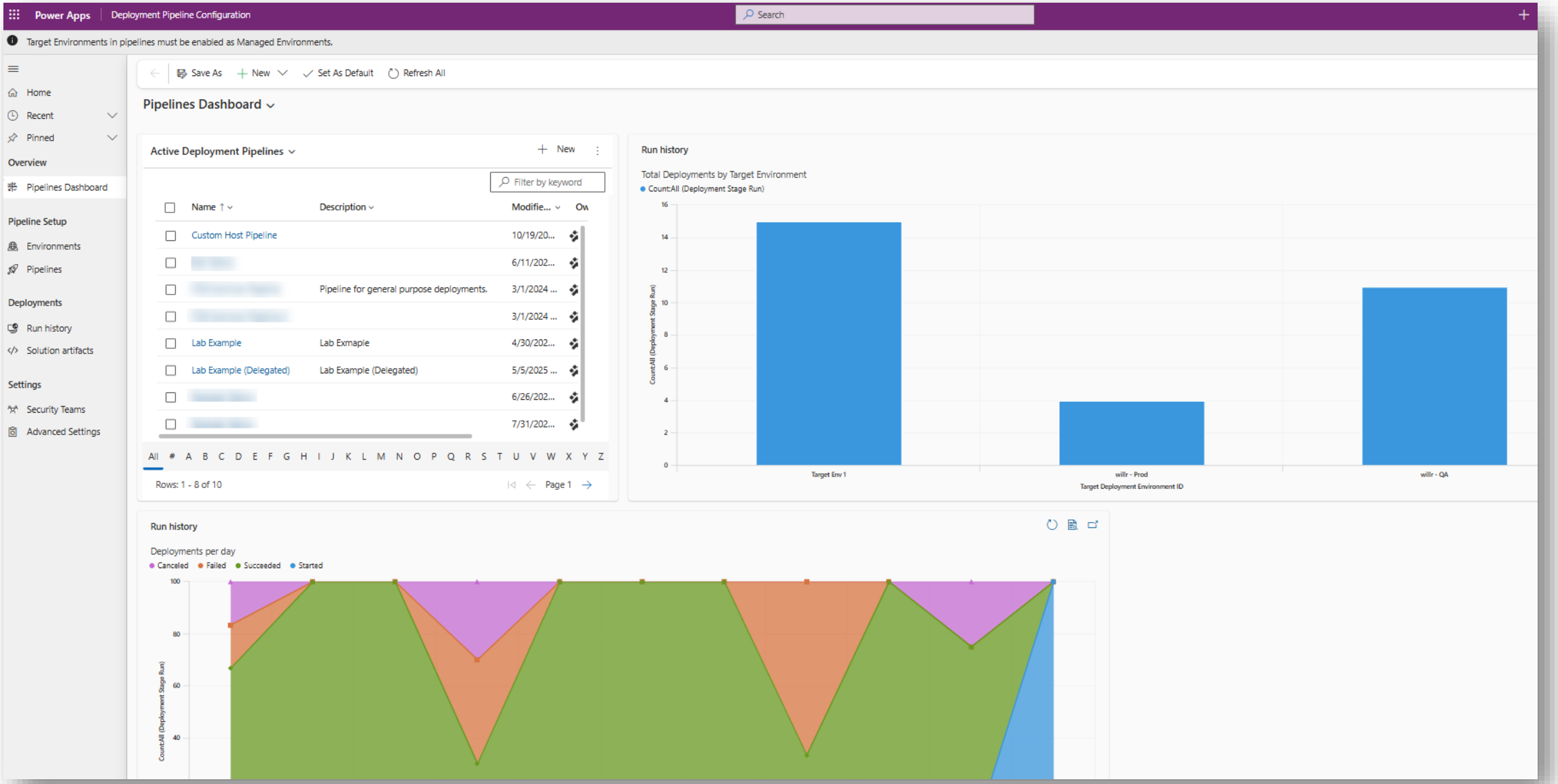
10 mins

Custom Host Pipelines

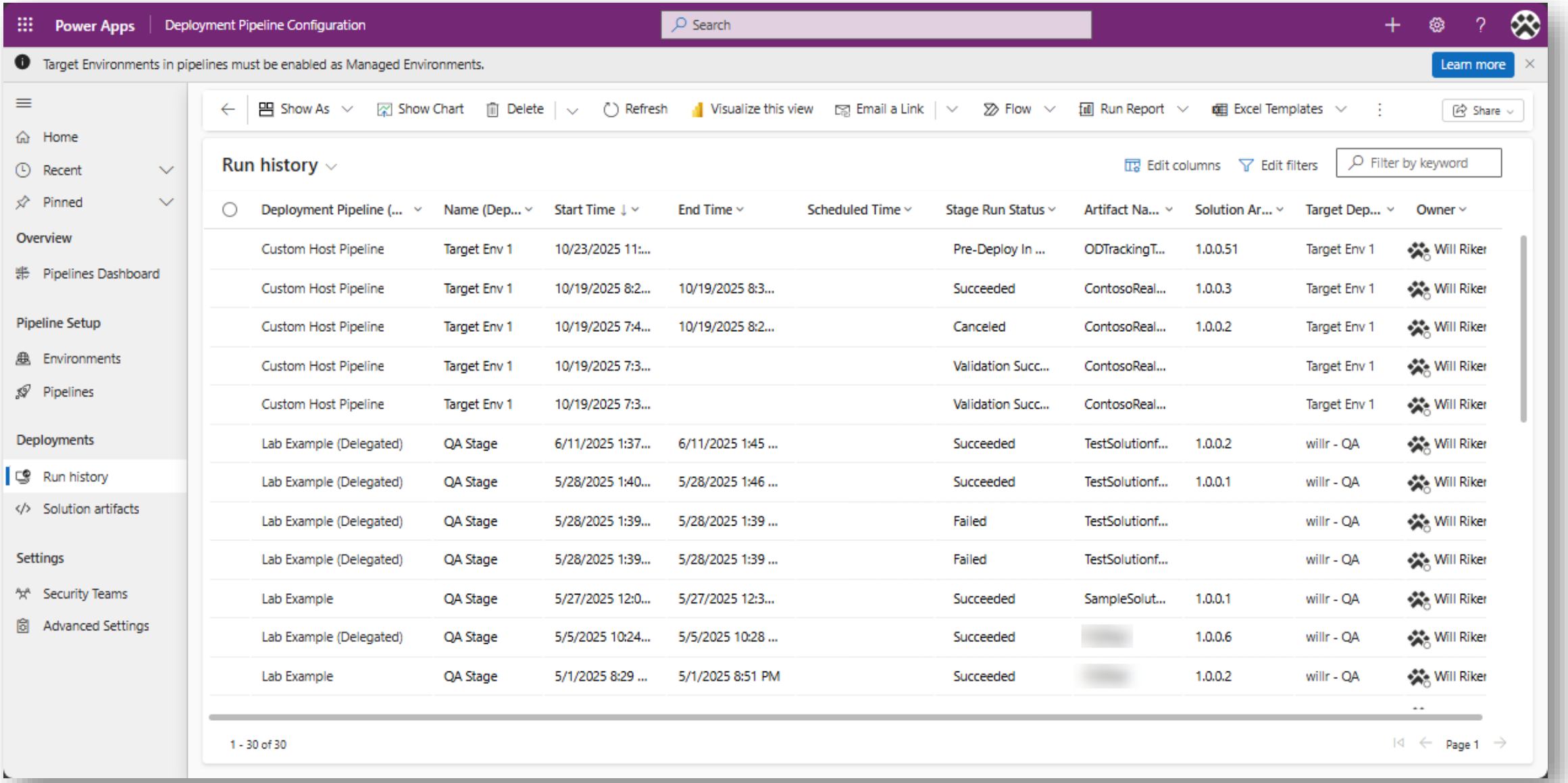
- Requires one dedicated Custom Host environment
- Managed from **Deployment Pipelines Configuration** app
- Supports up to **seven** deployment stages
  - At least one **Source** environment (usually Dev env)
  - Up to seven **Target** environments (UAT, Staging, Pre-Prod, Prod, etc.)



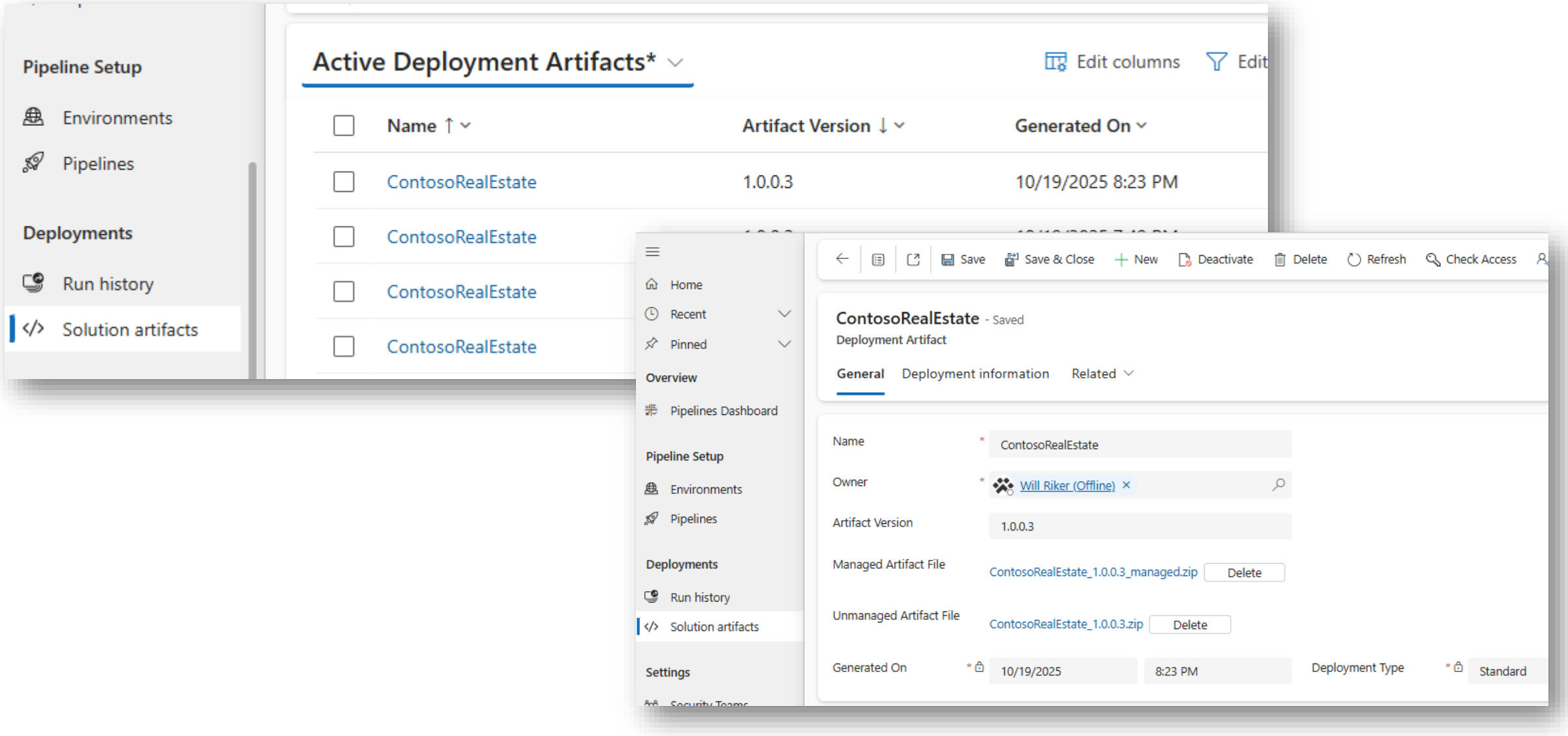
Deployment Pipeline Configurations app



Deployment Pipeline Configurations app



Deployment Pipeline Configurations app



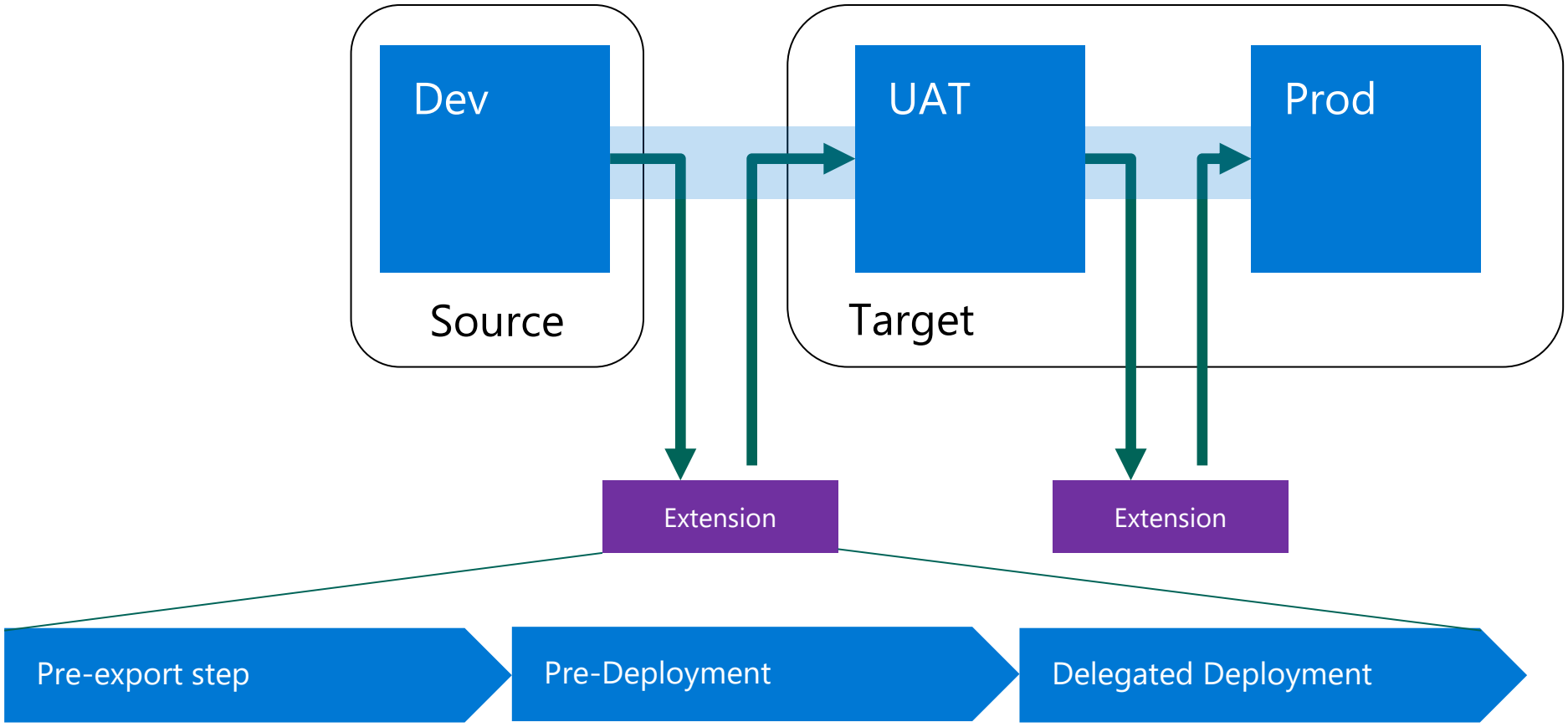
Microsoft Power Platform

Create a Custom Host Pipeline

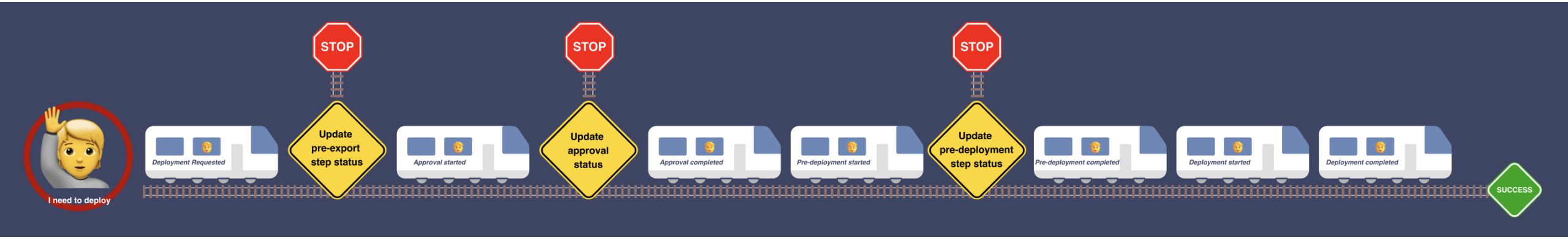
Lab 3 – Task 2

20 mins

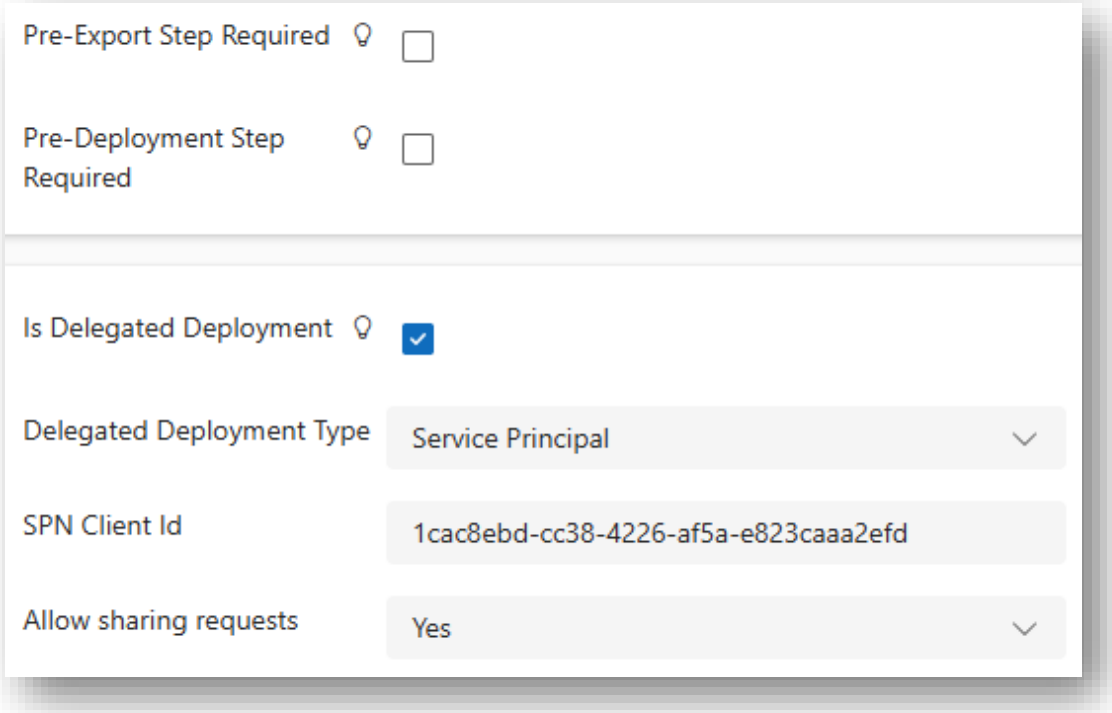
Extending the Custom Host Pipeline



Power Platform Pipelines Extensibility

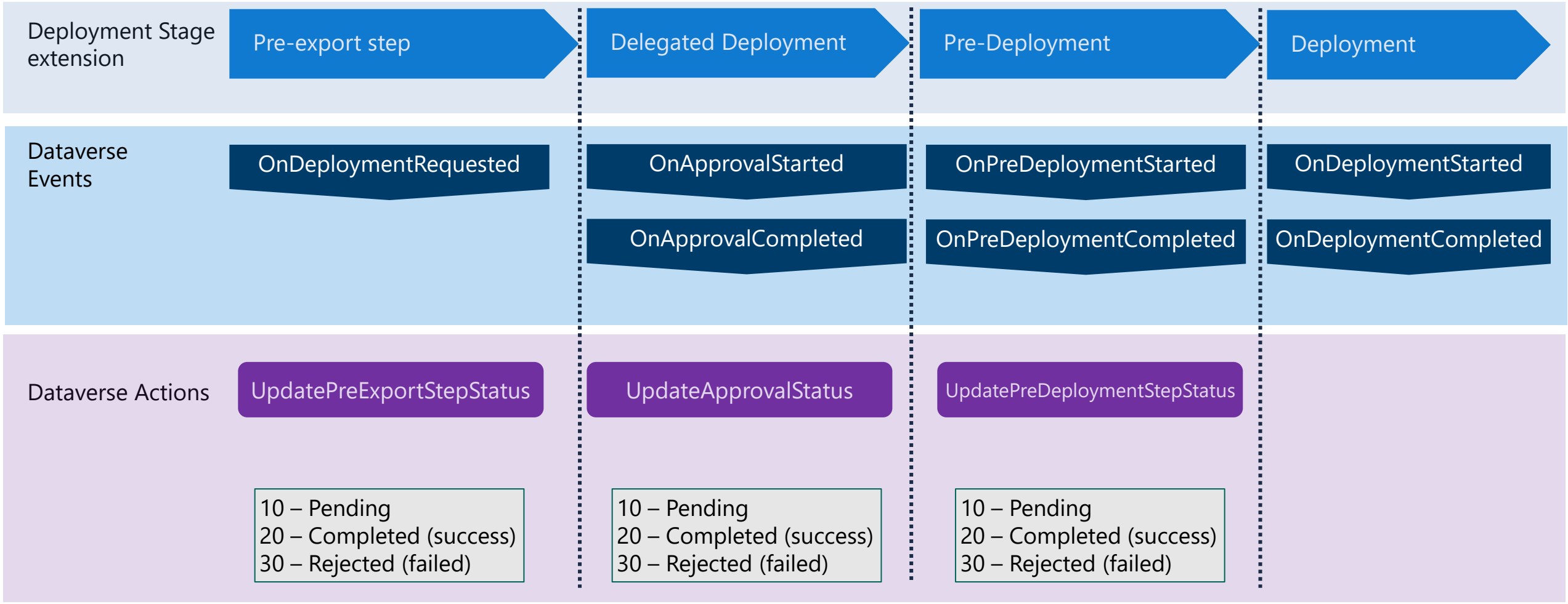


- Pipelines deployments progress through multiple predefined steps until deployment completion. Gated extensions insert custom steps into the progression where custom business logic can be executed.
- **Pre-export Step Required** allows running custom validation logic when a deployment request is submitted
- **Pre-deployment Step Required** provides additional flexibility to insert a custom step after a deployment is approved. For example, if a final approval is required
- **Is Delegated Deployment** carries out deployments using a service principal or pipeline stage owner's identity instead of the requesting maker's.



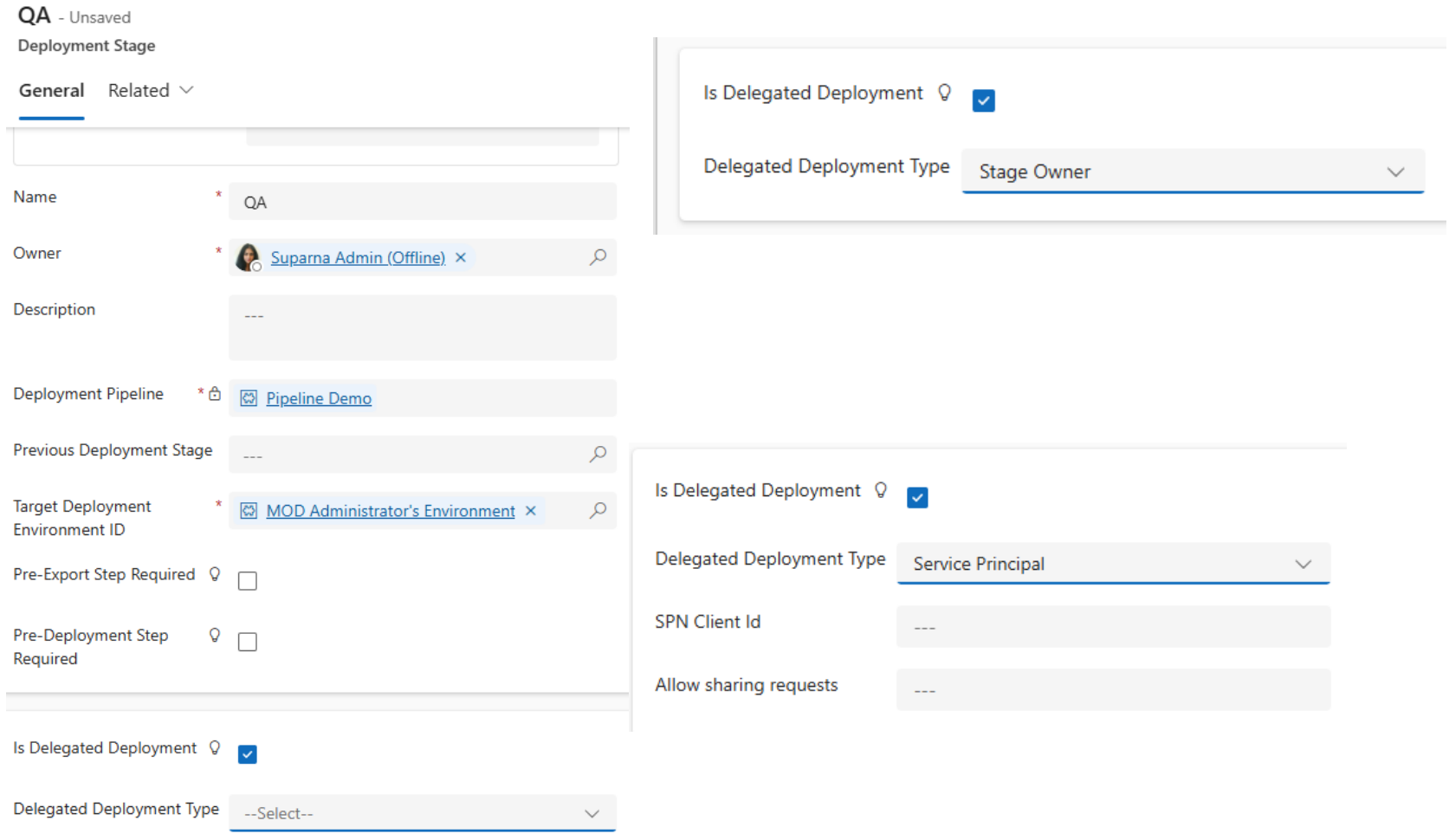
Extensions

Each of these Extension events maps to a Dataverse event



Pipelines with Delegated Deployment

- Delegated deployments can be run as a service principal or pipeline stage owner.
- When enabled, the pipeline stage deploys as the delegate (service principal or pipeline stage owner) instead of the requesting maker.
- For deployment with Stage Owner, Configuration is more straightforward compared to service principals, but **solutions containing connection references for oAuth connections can't be deployed.**



Microsoft Power Platform

Extend Pipelines (Optional)

Lab 3 – Task 3

40 mins

Microsoft Power Platform

Redeploy a prior version

Lab 3 – Task 4

5 mins

Microsoft Power Platform

Lunch 11:45 – 12:45

Agenda

Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

Microsoft Power Platform

Lab 4: Lifecycle mastery in Power Pages

Microsoft Power Platform

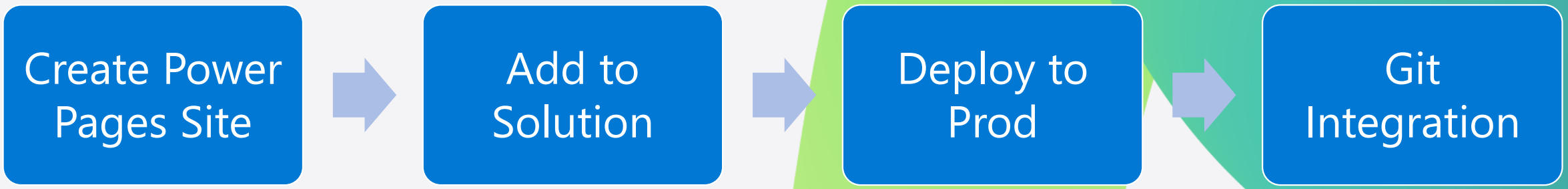
Introduction

Agenda

Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

Microsoft Power Platform

Lab Overview



Agenda

Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

Microsoft Power Platform

ALM Best Practices

Join the Power Platform DevOps community on LinkedIn!

https://PP-DevOps-LI

What’s the first step I should take?

Start with preferred solutions and Pipelines

Anyone can succeed without the underlying complexity

As you advance, learn how it works

Then add Git and developer tools as needed

What are the recommended best practices?

Avoid dependencies and layering issues

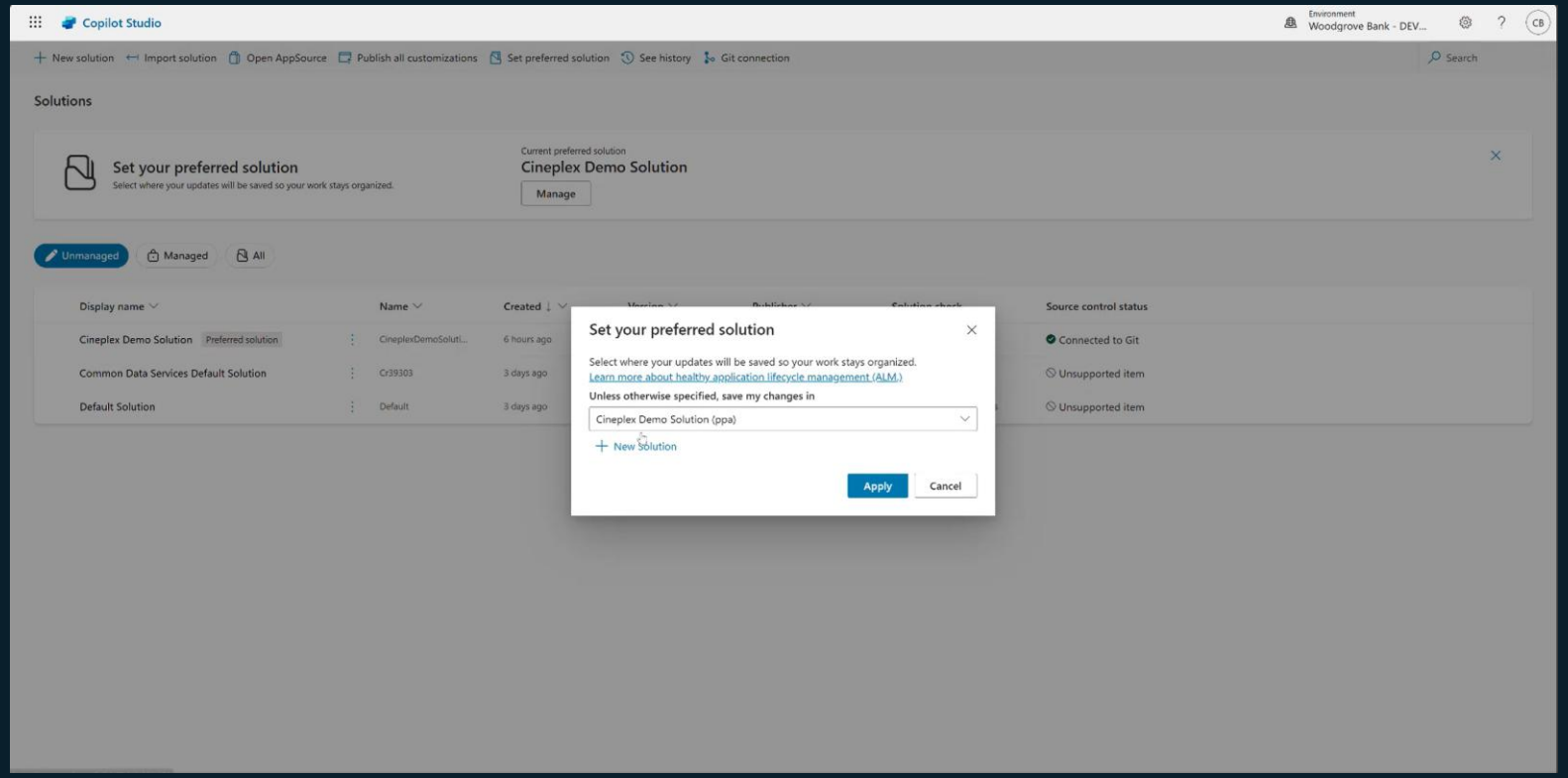
Only develop **1 unmanaged solution** per dev environment

If your project has > 1 solution, install others as managed in dev

Use a single solution publisher

Set a Preferred Solution

- Avoids dependency issues downstream
- Ensures you don’t deploy someone else’s work inadvertently
- Essential if you must share dev environments
- Per user, per environment



Use a single solution*

*unless components must deploy independently
e.g. a base solution used by multiple apps

Use a single solution publisher

Prefixes are stamped on everything

Prefixes are immutable

Lets you re-architect later

Deploy with a service principal

Least-privileged access

Keep production secure

Supported in Pipelines and dev tools

Enable “Block unmanaged customizations”

Blocks “dev in prod” and active layers

*First review component specific limitations

Avoid solution patches

They were invented for problems that no longer exist

Adds complexity and overhead without providing any value

Use Git branches for version control

Never add solutions to manage what ships

Use the same solution with different Git branches

Use Solution Checker

Catch issues before production

Gate pull requests in CI

Test and Evaluate Copilot Studio Agents

Public Preview

Automated testing for Agents:

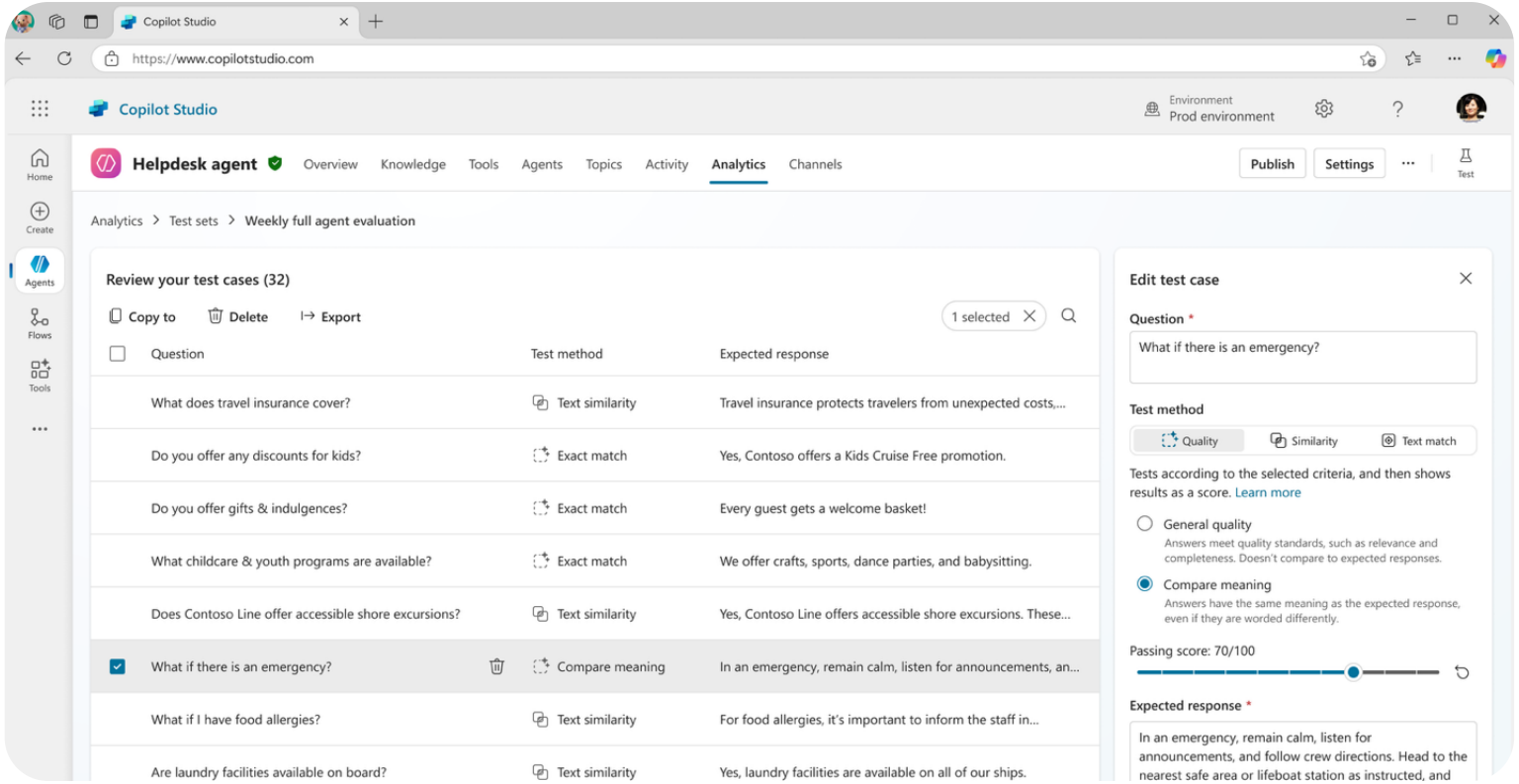
Structured, automated testing enables makers to evaluate agent quality before and after deployment eliminating need for manual, ad hoc testing.

Flexible, Scalable Evaluation Methods:

- Create evaluation sets using manual questions, imported files, get questions from prod data or even AI-generated queries
- Choose from multiple test methods to assess agent quality
- Pass/fail indicators with detailed scores and insights
- Compare results across runs to track improvements and detect regressions.

Continuous Improvement & Risk Reduction:

Identify issues early, reduce risk, and maintain quality as agents evolve. build → test → improve.



Use to gate deployments

aka.ms/CopilotStudioEvals

How do I manage team development effectively?

Healthy team ALM

Source control is the truth

Ship managed to test & prod

Non-prod environments are disposable

Automate everything

Develop in isolation

Defined approvals, hotfix, & rollback

Connect Git only where you build

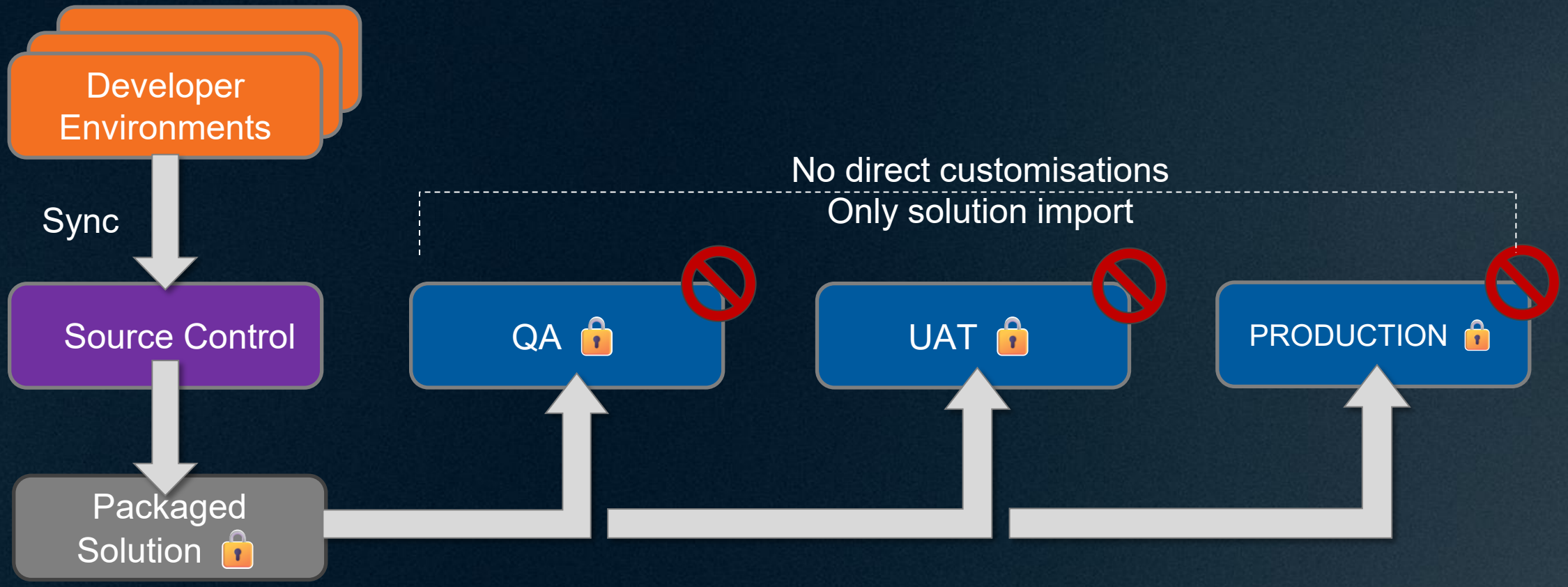
Git connected

- Feature dev
- Hotfix

Pipeline-only (managed)



Source Control is the single source of truth
across time, versions, and environments



Deployment guardrails

Build from source, not environments

Version every artifact

Protect main; require reviews

Smoke test every deployment

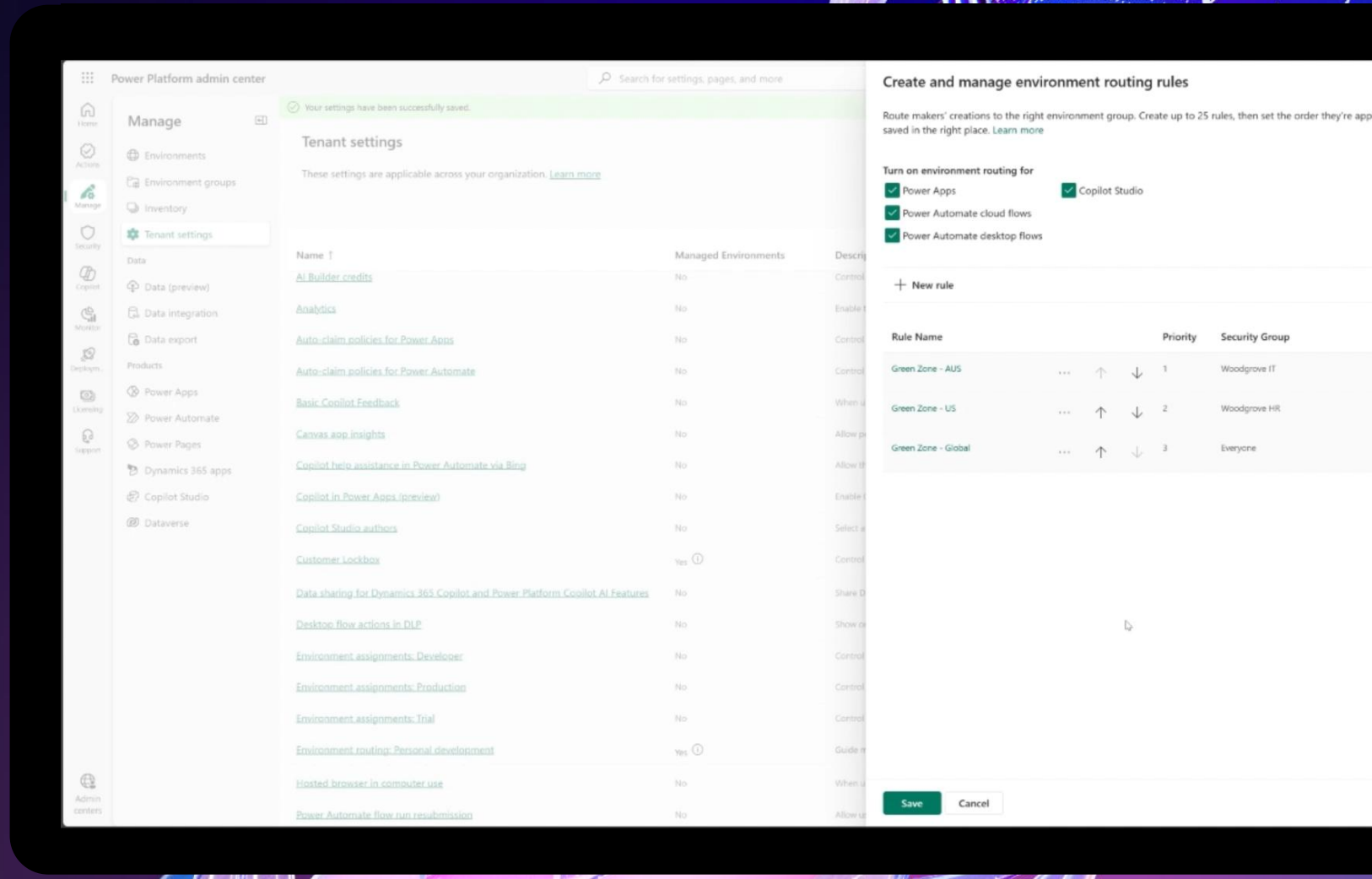
As an admin, how do I scale the process org-wide?

Environment routing

Generally available

Direct users to developer environments **instead** of the **default environment**.

- Supports Copilot Studio, Power Apps, Power Automate Cloud and Desktop, and **Copilot Managed Runtime**
- Designed to support simple to complex organizational structures.
- Attribute-based conditions via security groups or based on user properties such as geography, or department.
- Priority-based rule evaluation

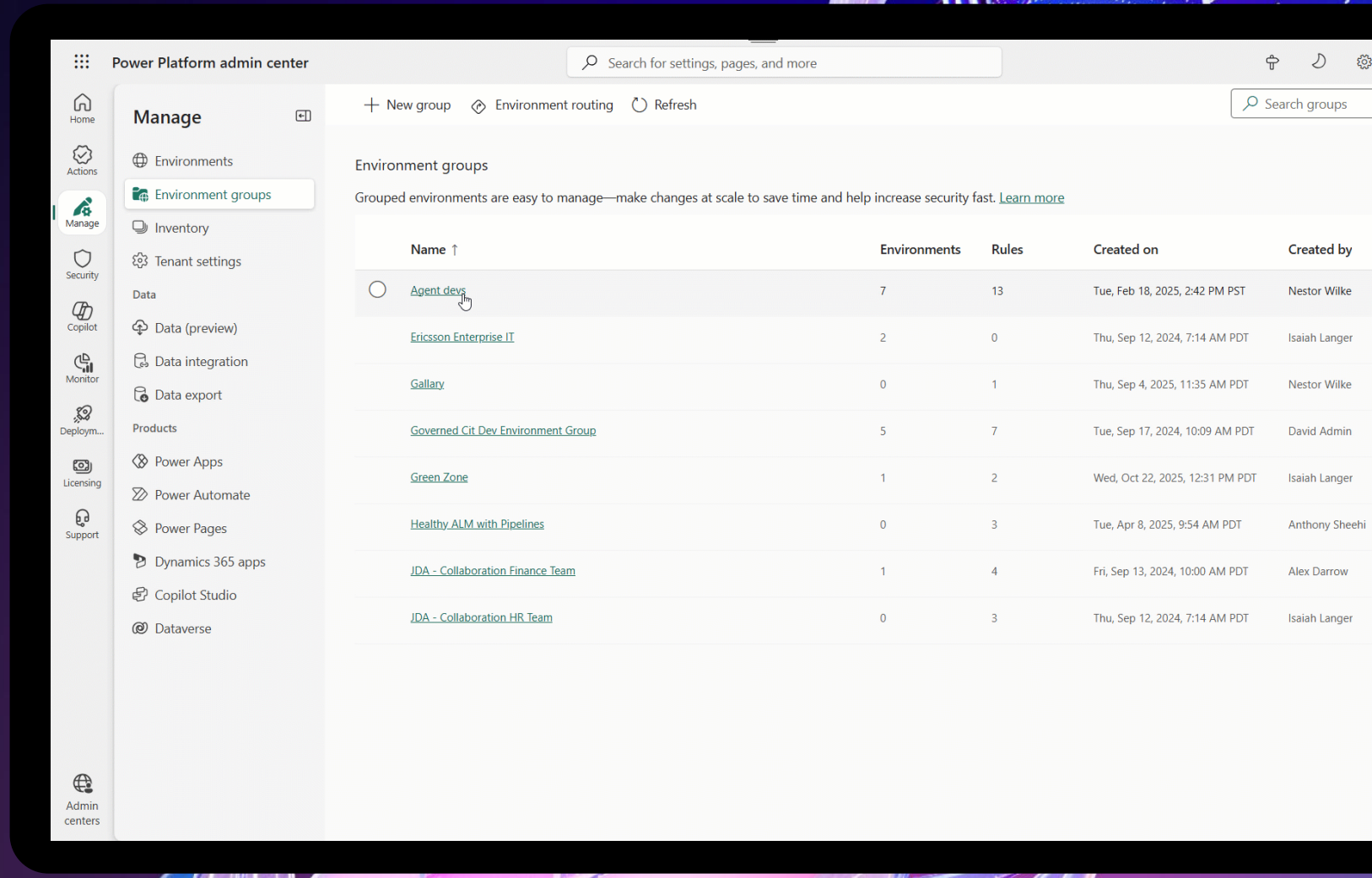


https://aka.ms/Environmentrouting

Default pipeline in Environment grups

Generally Available

- No more manual pipeline management
- Makers and their development environments are automatically onboarded to the default pipeline
- Allows different configurations for different groups

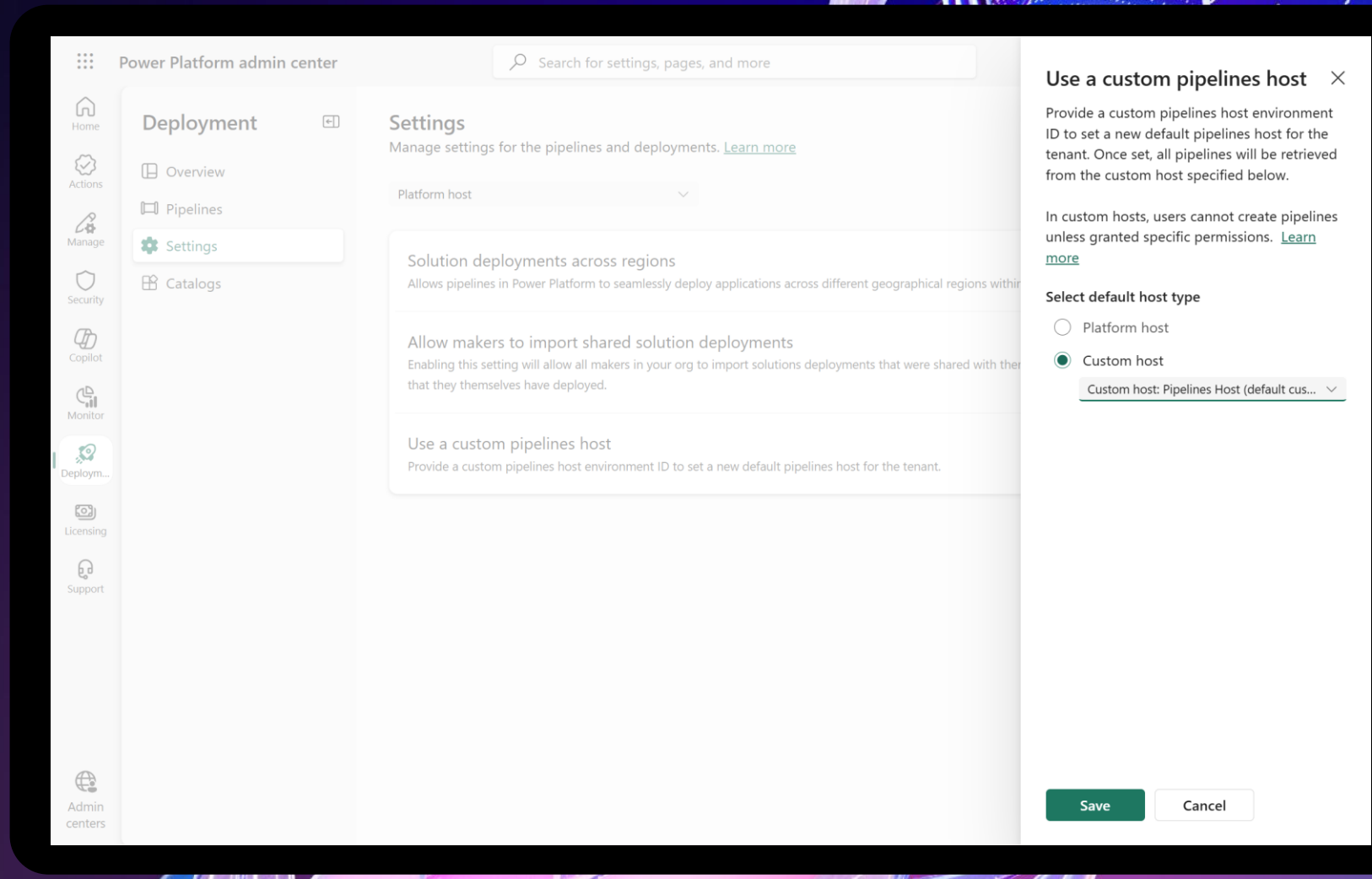


https://aka.ms/default-pipeline

Default custom host for Pipelines

Generally Available

- Central Pipelines administration for the tenant
- Replaces the platform host with a custom host
- Full control over security and governance (unlike the platform host)



Deployment hub

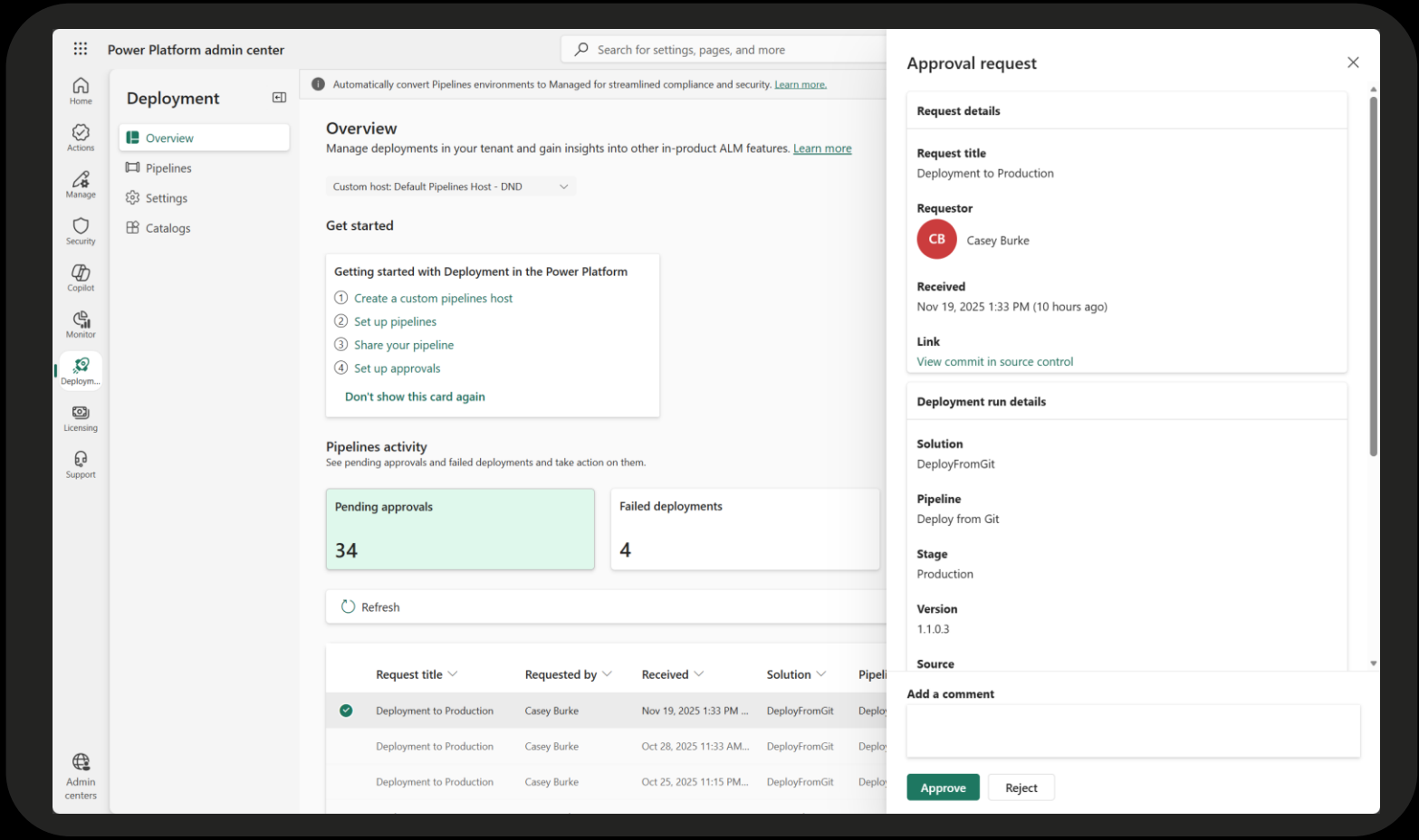
Generally available

Manage the DevOps process tenant-wide

Central visibility and control

Debug and retry failed deployments

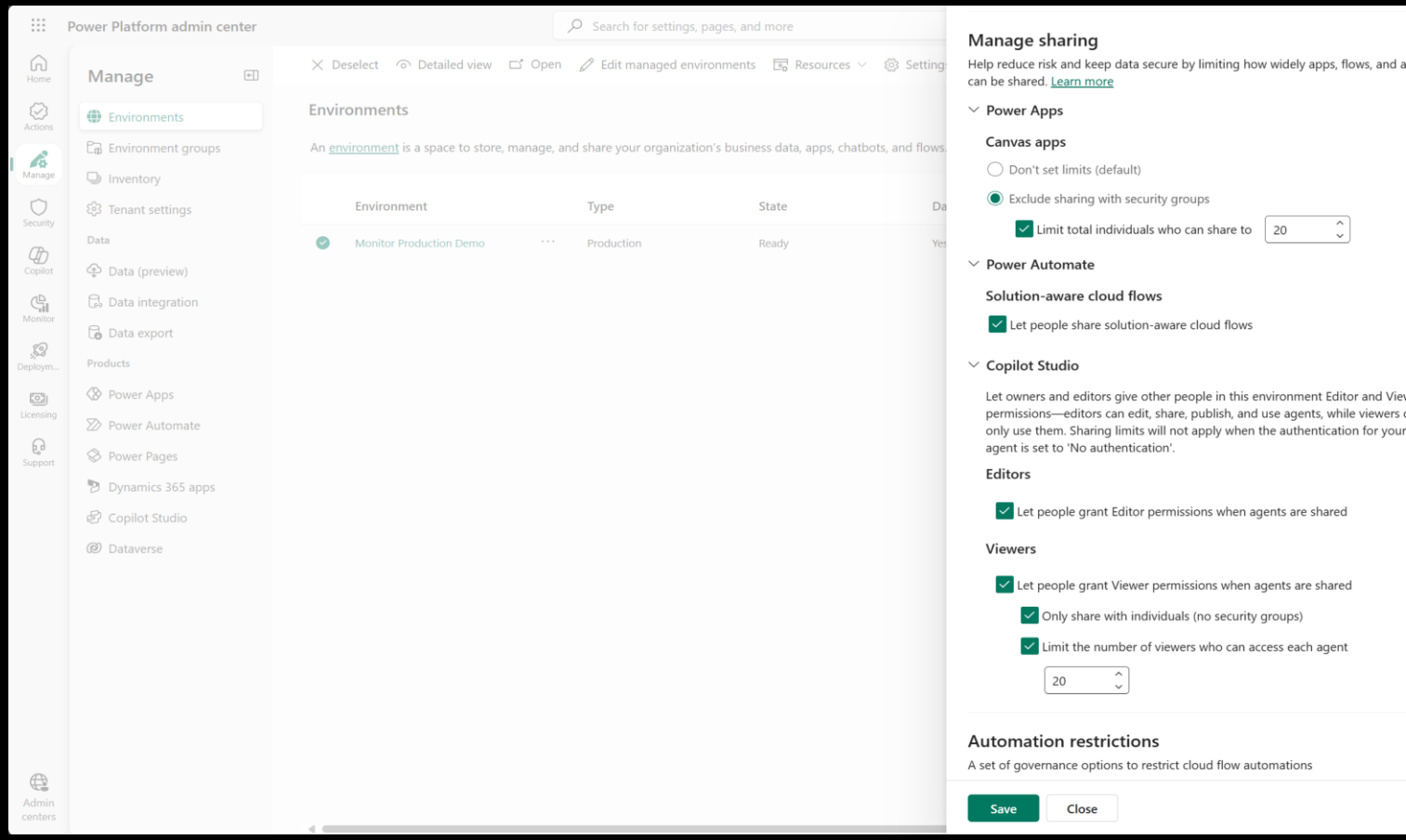
Enhanced deployment approvals, “host” creation, and compliance and automation settings



Set sharing limits in Dev and Test

Generally available

- Prevent pseudo prod from oversharing in dev and test
- Available for Copilot Studio agents and Power Apps code apps, in addition to canvas apps and cloud flows.

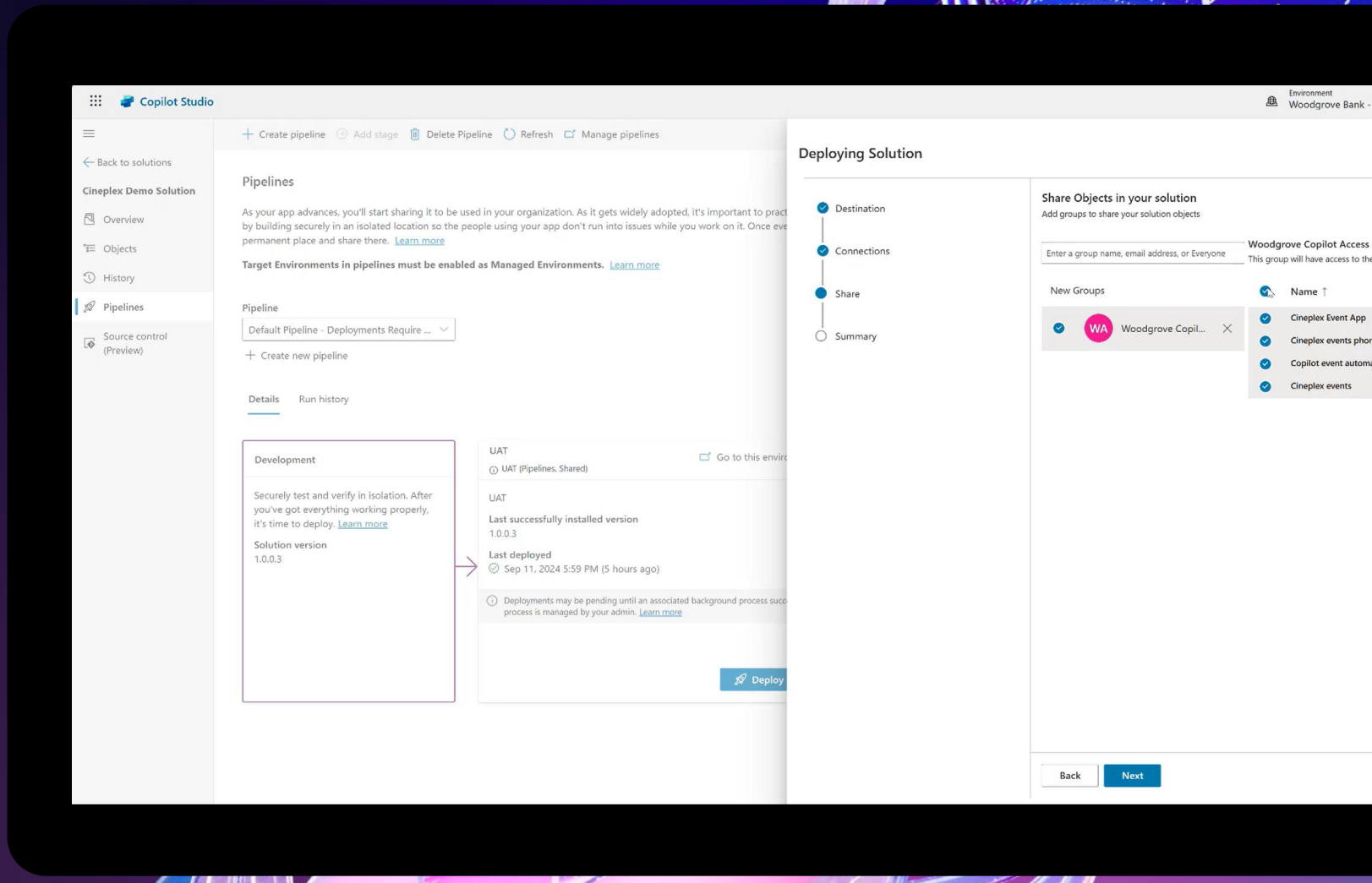


aka.ms/LimitSharing

Sharing during deployment

Generally available

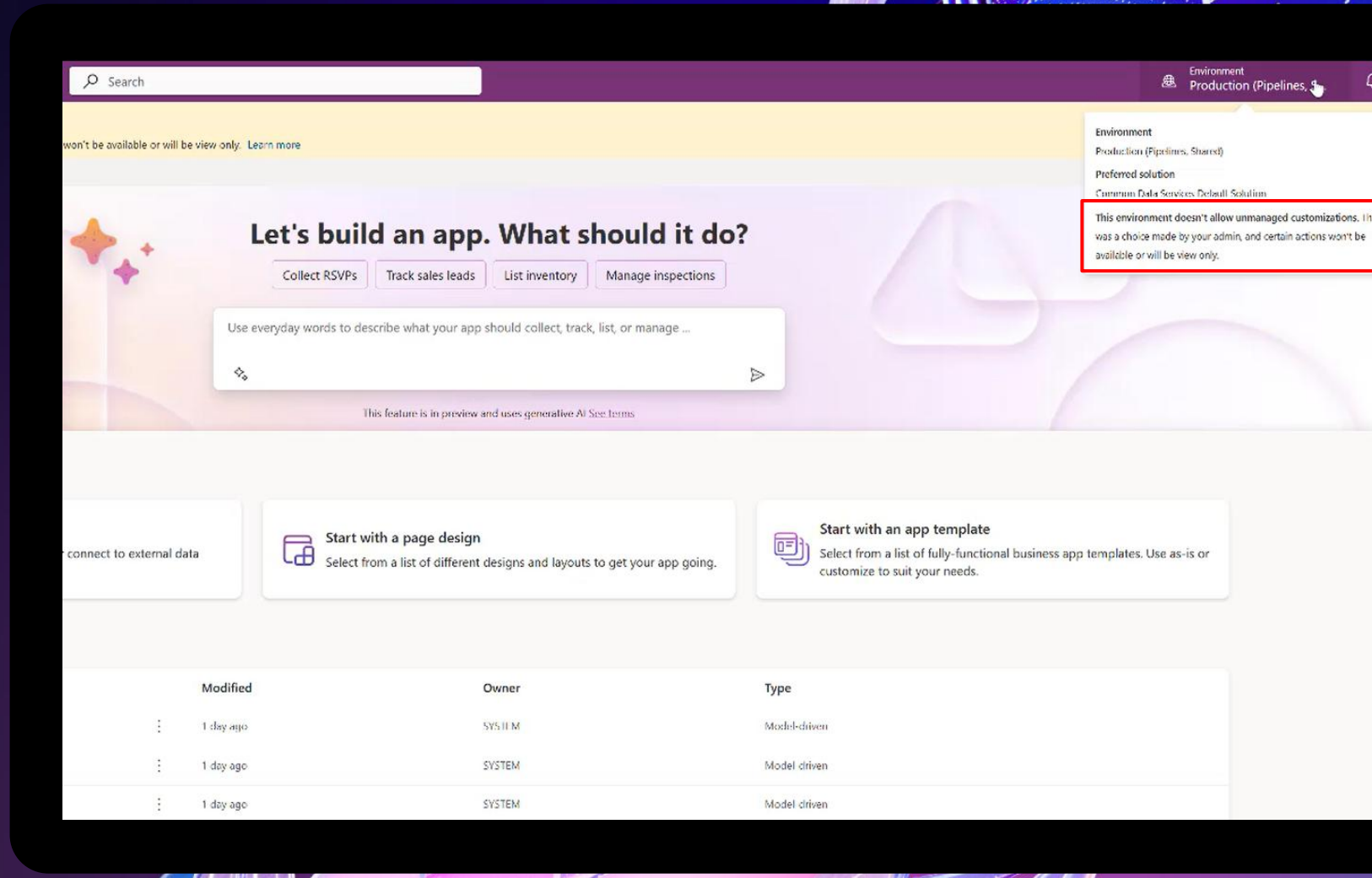
- Configure pipeline to run as service principal (avoids makers needing privileged access)
- Makers request sharing
- Review and approve or reject deployment + sharing request
- Supports MCS agents, canvas apps, flows, and security roles



Block unmanaged customizations

Generally available

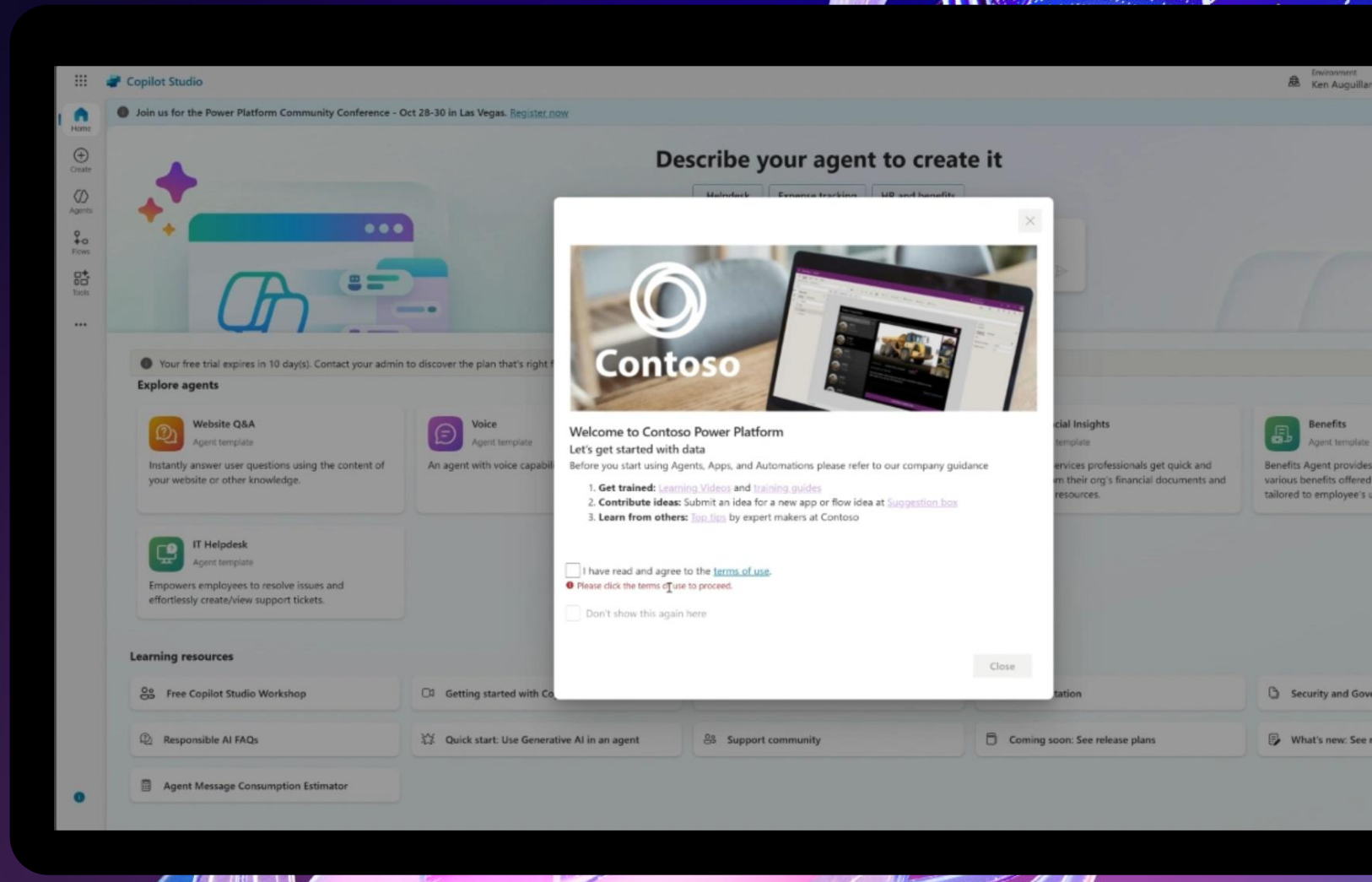
- Blocks dev in prod
- All changes are reviewed, versioned, approved, then deployed
- *Review known limitations before enabling broadly



Maker onboarding

Generally Available

- Centralized onboarding experience
- Ensures **consistent communication of policies, and guidelines** across environments.
- “Read and Understood” functionality — also known as: User Consent, or Terms of Acceptance.

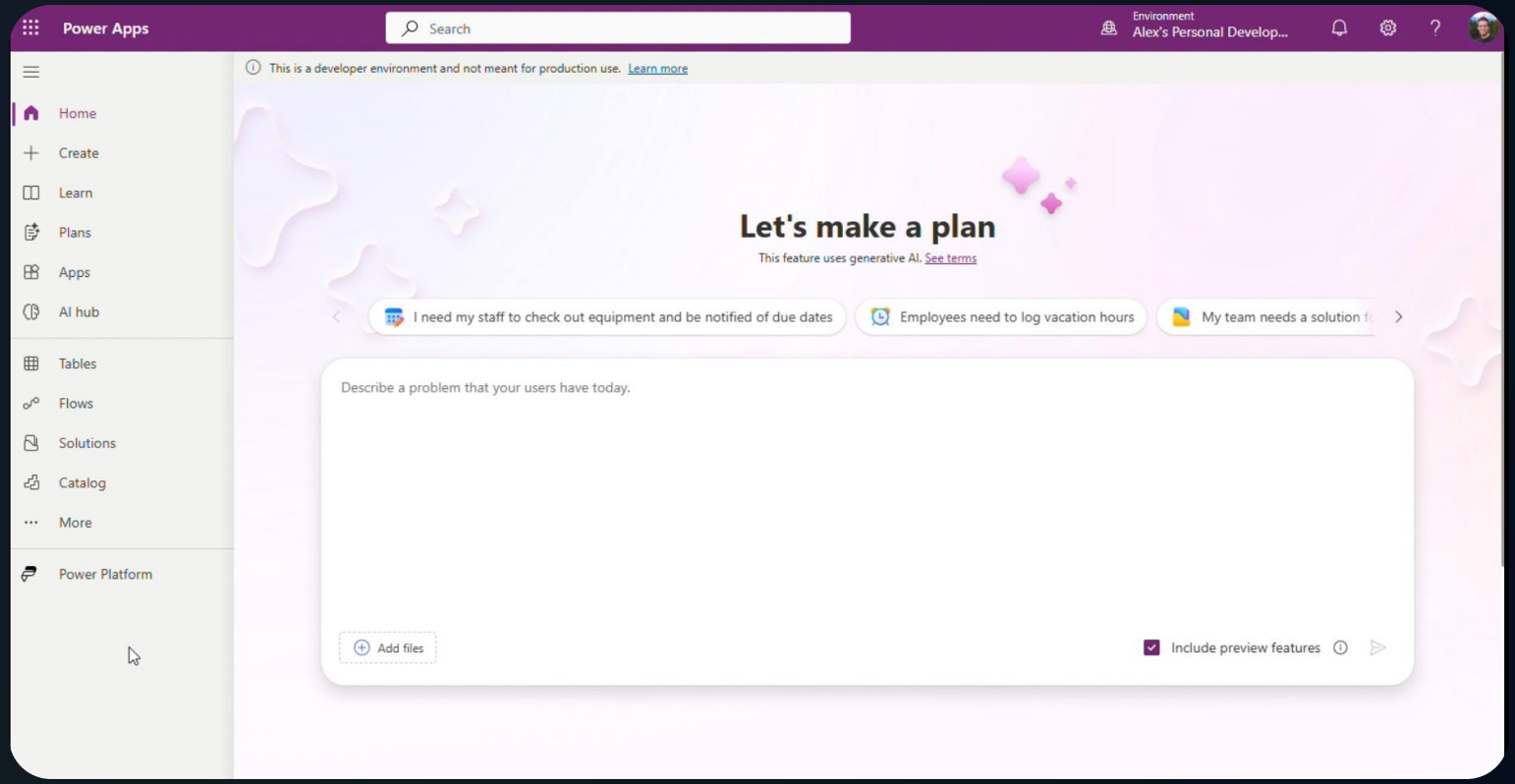


https://aka.ms/makeronboarding

Distribute reusable assets with Catalog

Generally Available

- Securely distribute reusable components at scale
- Full submission, approval, governance, and acquisition lifecycle tools
- Mass distribute agents, automations, and more
- Acquisition friendly for non-technical users

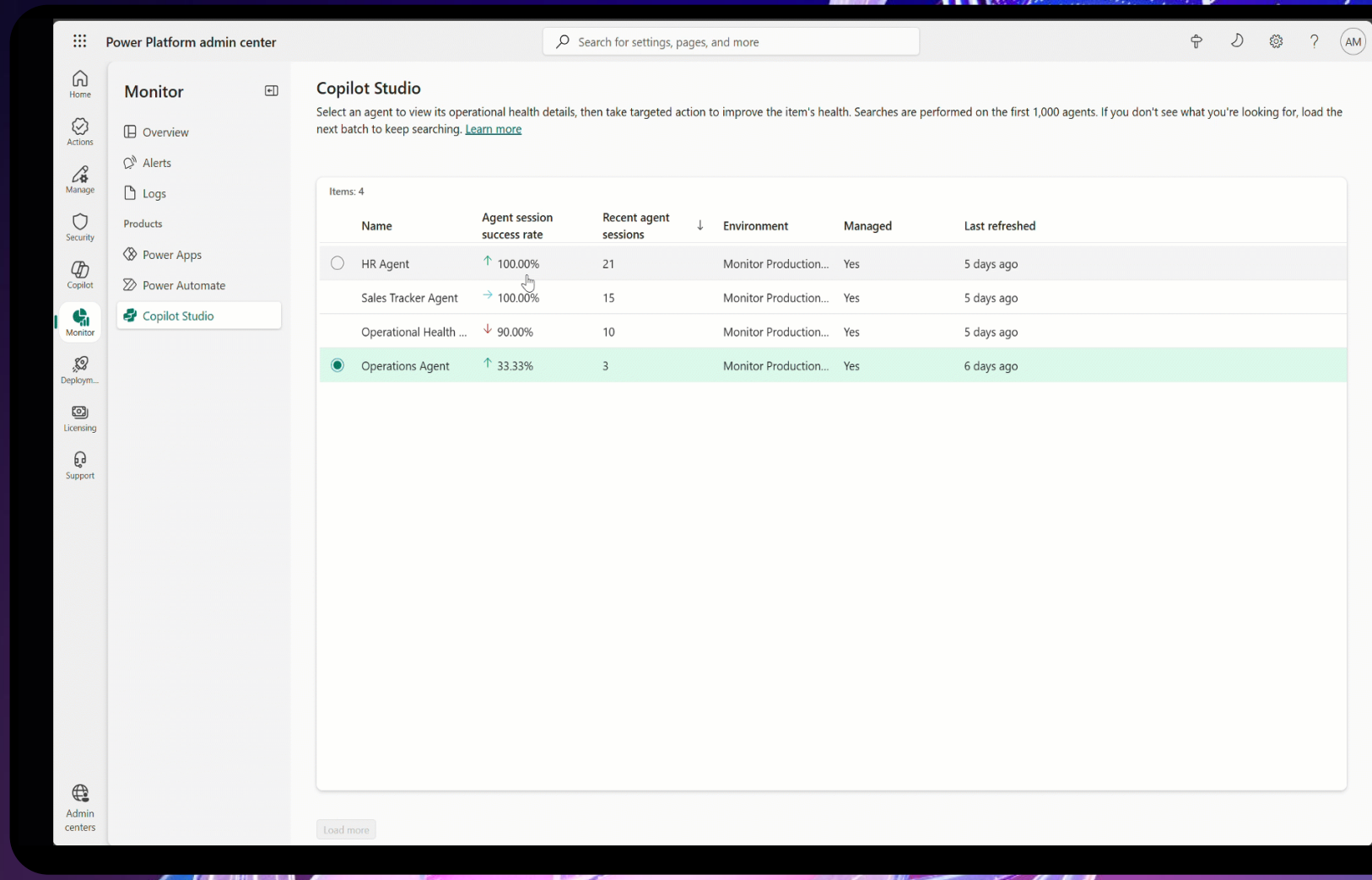


aka.ms/Power-Catalog

Monitor health of your portfolio

Generally Available

- **Monitor health** of apps, agents, and automations
- Set alerts for degradations



aka.ms/MonitoringHub

Microsoft Power Platform

Copilot Managed Runtime ALM

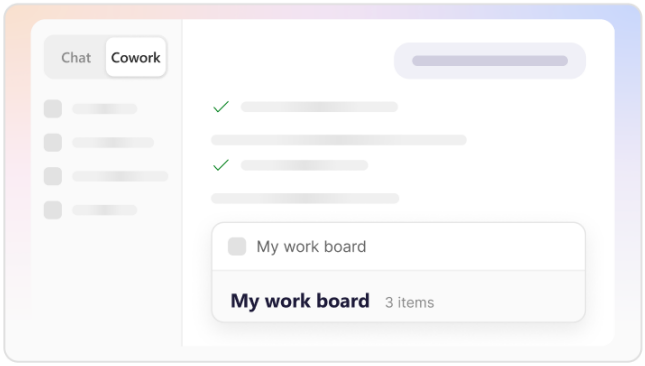
Agenda

Lab	Time	Topic
Presentation	9:00 AM	Solutions and DevOps fundamentals
Lab 1	9:30 AM	Setup and configure
Lab 2	9:45 AM	Fusion and Code-first development
Coffee	**10:00 AM**	**Coffee break**
Lab 2 (cont)	10:15 AM	Fusion and Code-first development
Lab 3	10:45 PM	Deploy
Lunch	**11:45 AM**	**Lunch**
Lab 3	12:45 PM	Deploy
Lab 4	1:30 PM	Lifecycle Mastery in Power Pages
Presentation	2:30 PM	Copilot Managed Runtime ALM
Presentation	3:00 PM	Best practices & Q&A
End	**4:00 PM**	

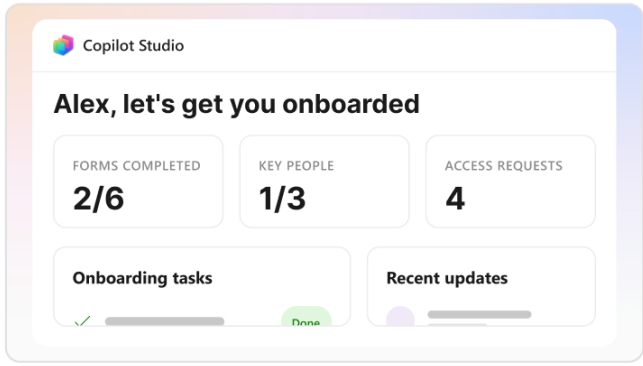
Microsoft entry points

There are four Copilot entry points for making artifacts and hosting with Copilot Managed Runtime

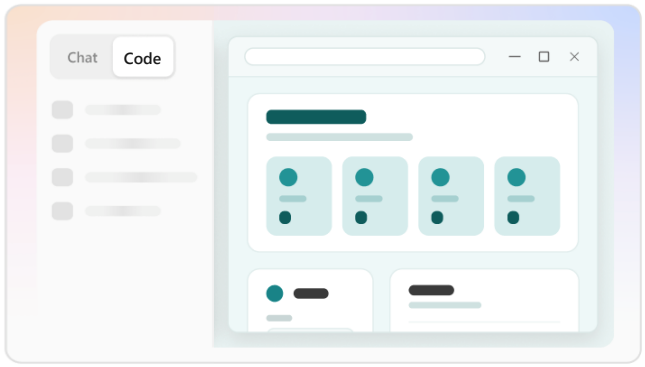
Copilot Managed Runtime host



Copilot Cowork



Copilot Studio



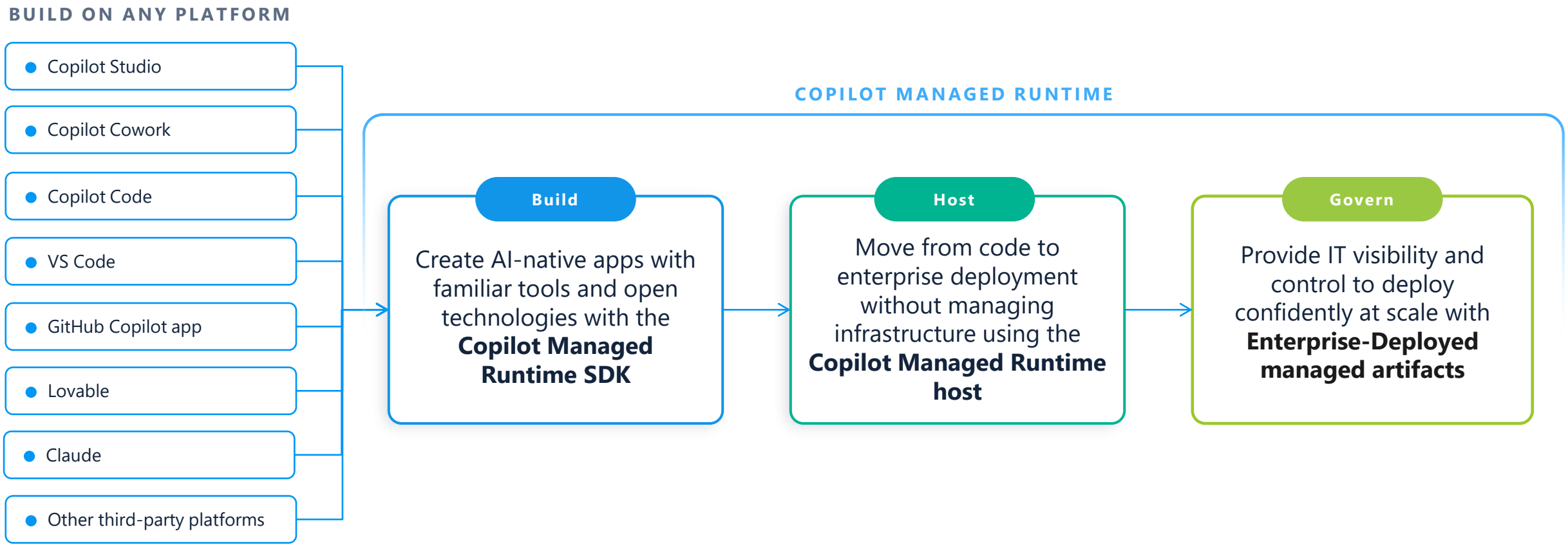
Copilot Code



GitHub Copilot

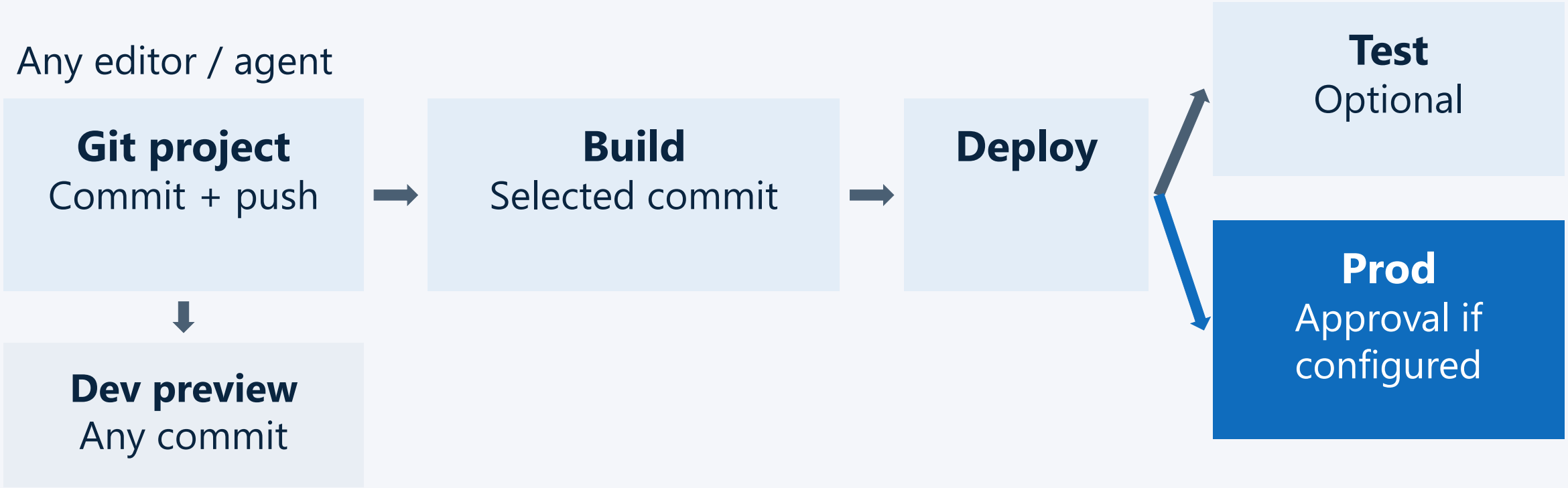
The platform

Connecting your platform and enterprise deployment. Build on any platform. The SDK enables a seamless handoff to the Host and enterprise deployment.



Build anywhere — the SDK connects — artifacts become hosted → an enterprise deployed artifact

CMR uses standard Git-based CI/CD



An environment is for runtime and governance.
Git is the single source of truth for your app's code.

ALM aware from the start

From solutions and environments to Git-backed projects and stages

Power Platform ALM	Copilot Managed Runtime ALM	Maker gains
Pick your environment or use the shared Default environment	Default to individual personal environments	**LESS DECISIONS** Environment is one less thing to understand and pick
Dataverse solutions: layers + dependencies	Always backed by a Git repo, possibly GitHub; no Dataverse required for ALM	**LESS TO LEARN** Skip solution mechanics, build in your IDE of choice
Configure pipelines; add Git and testing	Repo and build/deploy wired together; BYO tools optional	**LESS TO SET UP** Built-in ALM
Move solutions across Dev / Test / Prod environments	Deploy commits through stages; preview, then explicitly go live	**CONTROL RELEASES** Choose when to go live

Environments remain governance boundaries. Configuration lives in projects with stage overrides.

CMR adds a path—not a replacement.

Solutions + pipelines

Dataverse +
solution-aware assets

Power Apps code apps, too

FIRST LOOK

Copilot Managed Runtime

Git-backed projects
for CMR-hosted apps

Use tools where you work.
Managed hosting, connectivity and governance.

Every Project has a Git Repo

Public Preview

OOB repos used by default

OOB repos are the default

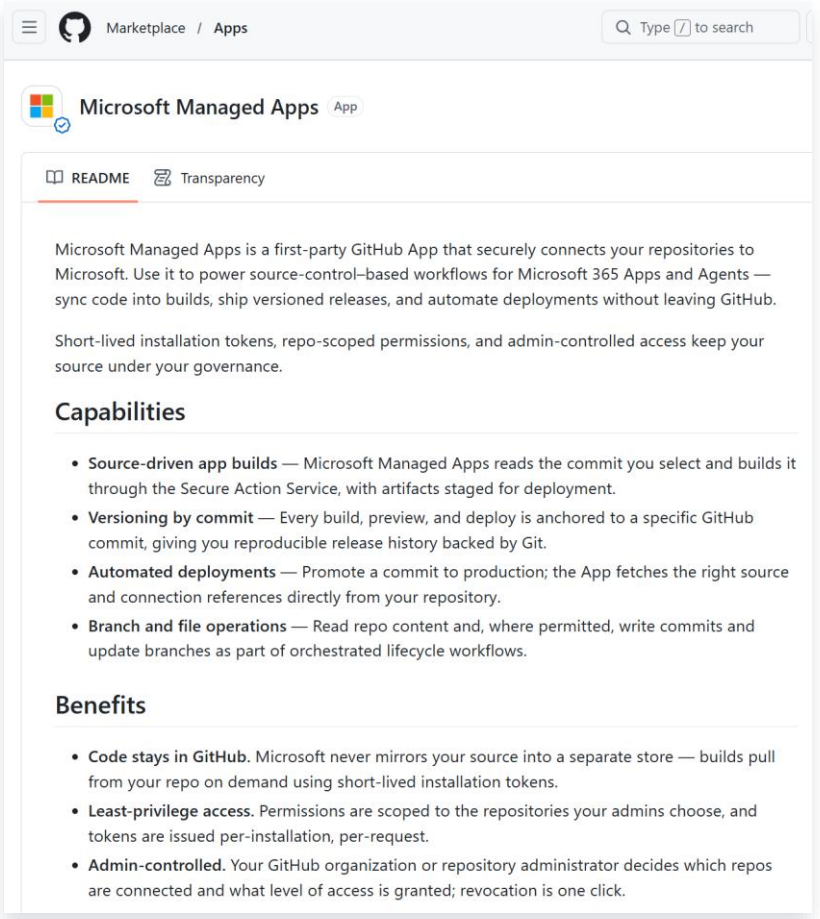
- Everything is version controlled within a remote Git repo (can be cloned locally)
- Source code is your single source of truth for development. Not solutions; not a Dataverse DB

GitHub Enterprise native integration

- Optionally host code in your own remote GitHub repos
- It's automatically wired into the platform's build-and-deploy pipeline

Bring-Your-Own

- Want more flexibility? Manage the E2E build and deployment with your Git system of record
- Supports service principal (SPN) auth for autonomous workflows



Three shifts from the ALM we know today

The unit of work, where stages live, and what a personal environment is allowed to host.



Projects

A project is a Git repo holding the apps, agents, and data that build and deploy together. Git repo can be in GitHub.



Single and Multi Environment Deployment

Every environment can host test and production stages on its own, as well as deploying to other environments if needed.



Personal Environment can host Prod

A personal environment can host a production app for the long term.

Single environment is the default path; multi-environment ALM stays available for advanced cases.

A project is one repo that builds and deploys as a unit

PROJECT = REPO

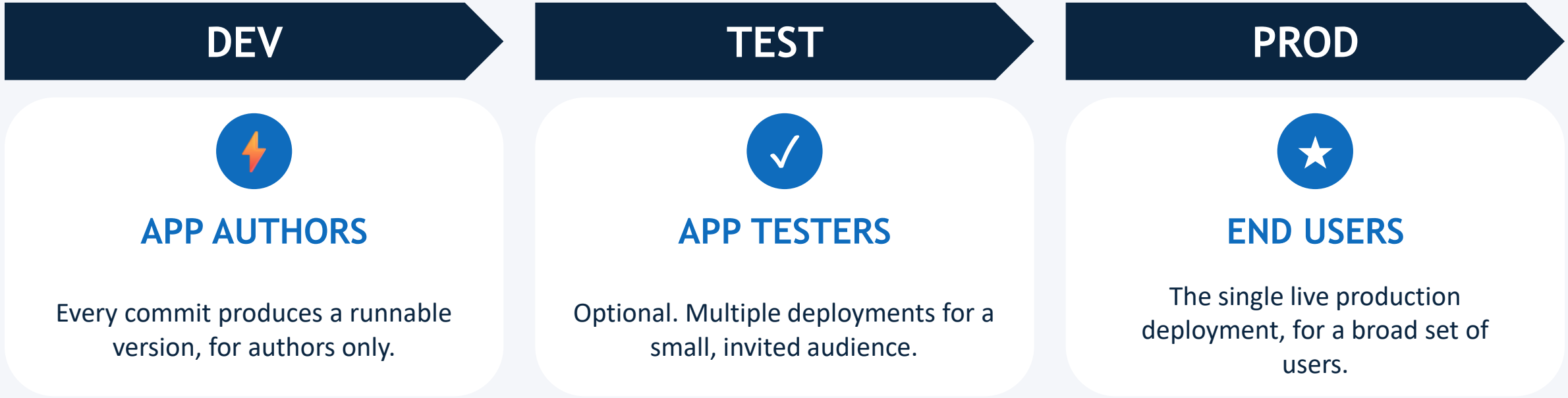
```
my-project/
   ms.project.config.json
   driver-app/
      ms.config.json
   dispatcher-app/
      ms.config.json
```

One directory per component, each with a config file.

We will continue to list apps independent of projects – makers and end users do not need to think about projects.

Select portions of config files can be overridden for a specific deployment.

Projects support three ALM “stages”



Deployments

- Run local build
- Run cloud preview

- Deploy to prod, the default target
- Create additional targets and deploy
- Run from a target with user permissions

Using a single environment removes friction

From environment- and solution-centric deployment to project-, Git-, and stage-centric lifecycle management

	Traditional Power Platform ALM	Managed Apps ALM
Primary unit	Dataverse solution	Git-backed project
Lifecycle topology	Dev → Test → Prod environments	Dev → Test → Prod stages
Source control	Optional or added to the process	Native and expected
Deployment motion	Export / import solutions	Deploy specific project commit
Environment role	Lifecycle + governance boundary	Governance boundary
Configuration	Environment variables + connection references	Project configuration + target overrides
Dataverse relationship	Core ALM foundation	Independent; can coexist
Pipeline role	Core deployment mechanism	Optional orchestration / governance

Bottom line: traditional ALM moves solutions between environments; Managed Apps ALM moves project versions through stages.

Production needs a trusted path

AI makes building fast. Going live still needs a human in the loop.

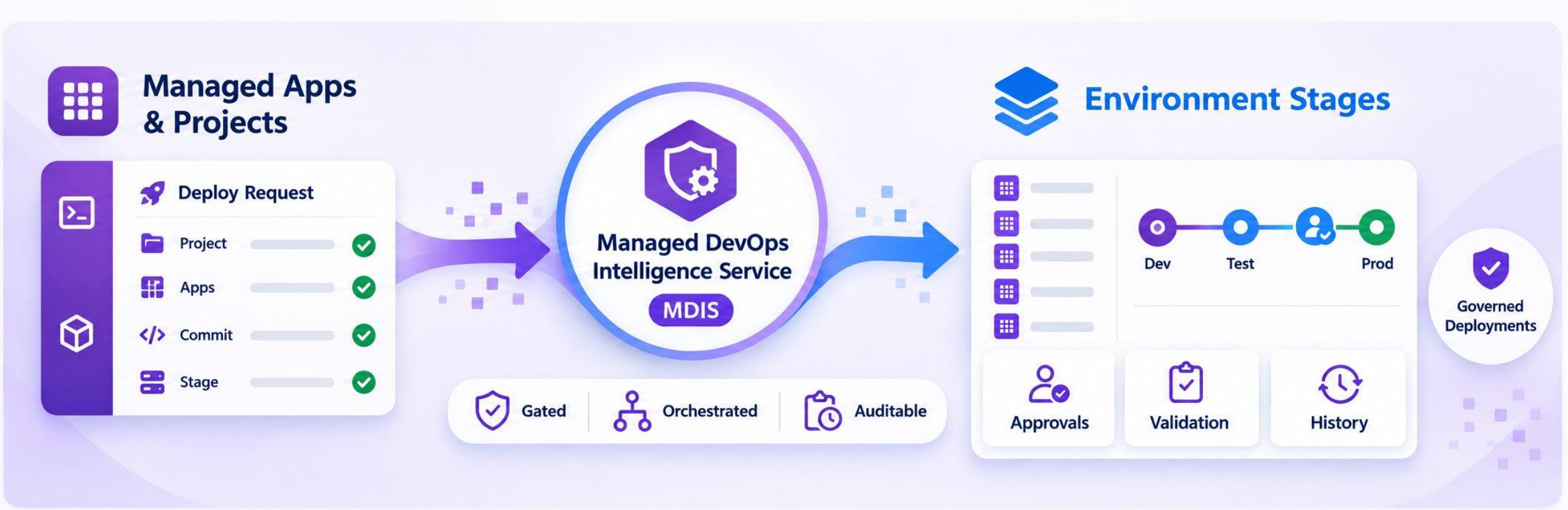
✓ **Who approved this change — and why?**

◎ **Can someone review it before users see it?**

= **Is what we tested what actually ships?**

✦ **Can agents deploy without us losing control?**

Deployment approvals



CMR deployment approvals

Coming soon

ROLES

PROJECT OWNER

Turns on approval by assigning approvers

```
ms project add approval <email>
```

Approval is required while at least one approver is assigned.

CONTRIBUTOR OR AGENT

Requests the deployment

```
ms project deploy
```

Can deploy, but can't approve unless explicitly assigned by the owner or admin.

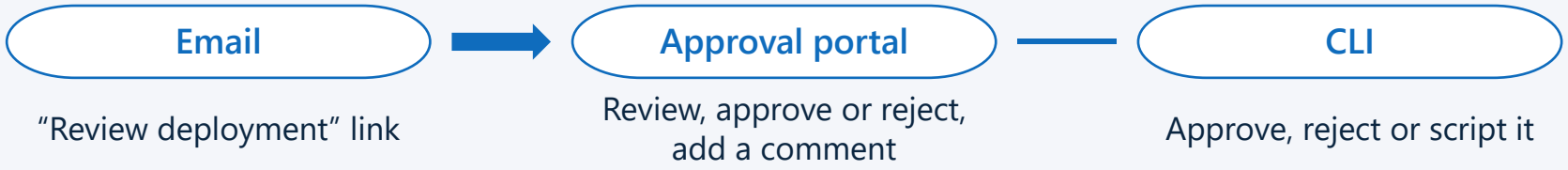
APPROVER

Reviews the commit and decides

```
ms deployment approve
```

Sees what's being deployed, without needing edit rights.

Approve deployments using the CLI or in-product experience



Demo CMR Deployment and Approvals

Microsoft Power Platform

Q & A

Microsoft Power Platform

Thank you

Join us Tues, Oct 28th at 2:15PM

MS250 Lifecycle Mastery: Managing Power Platform Assets in the Age of Agents

© Copyright Microsoft Corporation. All rights reserved.